

第三章 典型密码原语

学习要求：问题；典型方案（2-3 个，能体现研究趋势）；现状和进展。

课时：4 课时。

本章介绍承诺、零知识证明、茫然传输、秘密共享等基础密码原语，这些原语常被用于设计安全协议和辅助解决密码学应用问题。

3.1 承诺

在日常生活中，承诺无处不在。例如，预约打车成功后，司机和乘客之间就互相做了一个承诺。到了预约时间，乘客等车，司机接客，这就是在兑现承诺。

3.1.1 基本概念

一般而言，承诺涉及承诺方和验证方，它允许承诺方向验证方对一个秘密值做出承诺，承诺方后续会向验证方披露此秘密值。

承诺协议包含两个阶段：

- **承诺生成（Commit）阶段**：承诺方选择一个敏感数据 v ，计算出对应的承诺 c ，然后将承诺 c 发送给验证方。通过承诺 c ，验证方确定承诺方对于还未解密的敏感数据 v 只能有唯一的解读方式，无法违约。
- **承诺披露（Reveal）阶段**：也称为承诺打开-验证（Open-Verify）阶段。承诺方公布敏感数据 v 的明文和其他的相关参数，验证方重复承诺生成的计算过程，比较新生成的承诺与之前接收到的承诺 c 是否一致，一致则表示验证成功，否则失败。

承诺有两个性质：

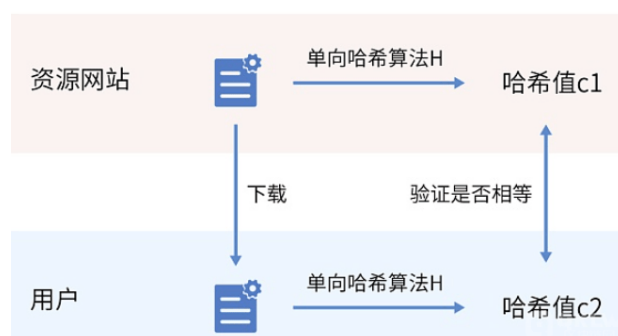
- **隐藏性（Hiding）**：在承诺方未向接收方披露承诺值之前，验证方无法得到与承诺值相关的任何信息，这一性质称为隐藏性。
- **绑定性（Binding）**：承诺方在对秘密值做出承诺之后，无法对秘密值进行任何修改，这一性质称为绑定性。

3.1.2 哈希承诺

哈希承诺就是一个简单的承诺协议。承诺生成阶段，将通过输入的消息 v 的哈希值 $H(v)$ 作为承诺并公开；承诺披露阶段，将对获取的消息 v' 通过计算它的哈希值 $H(v')$ 并与之前公开的承诺 $H(v)$ 进行比对完成验证。

基于单向哈希的单向性，难以通过哈希值 $H(v)$ 反推出敏感数据 v ，以此提供了一定的隐匿性；基于单向哈希的抗碰撞性，难以找到不同的敏感数据 v' 产生相同的哈希值 $H(v)$ ，以此提供了一定的绑定性。

典型应用。某些网站在提供下载文件时，会提供对应文件的单向哈希值。这里，单向哈希值便是一种对文件数据的承诺。基于下载的哈希承诺，用户可以对下载文件数据进行校验，检测接收到的文件数据是否有丢失或变化，如果校验通过，相当于网站兑现了关于文件数据完整性的承诺。



提升安全性的哈希承诺。对隐私数据机密性要求高的应用，需要注意哈希承诺提供的隐匿性比较有限，不具备随机性。对于同一个敏感数据 v ， $H(v)$ 值总是固定的，因此可以通过暴力穷举，列举所有可能的 v 值，来反推出 $H(v)$ 中实际承诺的 v 。

利用随机预言机模型可以构造更加安全高效承诺协议。如果想对 x 做出承诺，只需要简单地选择一个随机值 $r \in_R \{0,1\}^k$ 并公布 $y = H(x \parallel r)$ 。后续只需要简单地披露 x 和 r 即可。

3.1.3 加法同态承诺

上节描述的哈希承诺不具备同态特性，多个相关的承诺值之间无法进行密文运算和交叉验证，复杂的验证都需要承诺披露后才能进行，对于构造复杂密码学协议和多方安全计算方案的作用比较有限。同态承诺（Homomorphic Commitment）与同态加密概念相似，可以在公开的承诺上直接执行一定的计算，使得其与明文上运算后的承诺一致。

1. 基本定义

加法同态承诺是指具有加法同态性质的同态承诺，即给定承诺 $\text{Com}(x; r_x)$ 和 $\text{Com}(y; r_y)$ ，存在运算 $\oplus$ ，满足

$$\text{Com}(x; r_x) \oplus \text{Com}(y; r_y) = \text{Com}(x+y; r_x+r_y)。$$

可以这么解释，对于两个敏感数据 x 和 y 的承诺执行运算 $\oplus$ ，与两个敏感消息先执行+之后再生成的承诺是相同的，都可以在承诺披露阶段完成验证。

2. Pedersen 承诺

Pederson 同态承诺（简称 Pedersen 承诺）是由 Torben Pryds Pedersen 于 1992 年提出的一种承诺。目前，Pedersen 承诺主要搭配椭圆曲线密码学使用，具有基于离散对数困难问题的强绑定性以及同态加法特性。

（1）椭圆曲线

回顾一下椭圆曲线，椭圆曲线上的点可以进行加、减或乘（乘以整数，也称标量）运算。椭圆曲线上的点满足以下特性：给定一个整数 k ，其可以与曲线上的点 G 进行标量乘法运算，即 kG ，所得结果也是曲线上的一个点。给定另一个整数 j ， $(k+j)G$ 等于 $kG + jG$ ，即椭圆曲线上的加法和标量乘法运算保持加法和乘法的交换率和结合律。

在 ECC 中，因为椭圆曲线离散对数问题的存在，如果选择一个非常大的数字 k 作为私钥， kG 和 G 作为相应的公钥，那么通过公钥推导出私钥 k 几乎是不可能的。

（2）具体构造

初始化阶段：初始化椭圆曲线，并选择曲线上的两个点 G 和 H 。

承诺阶段：承诺方选择随机数 r 作为盲因子，对隐私信息 v 计算承诺值 $C = rG + vH$ ，并发送给接收方。

披露阶段：承诺方发送 (v, r) 给接收者，接收者验证 C 是否等于 $rG + vH$ ，如果相等则接受，否则拒绝承诺。

（3）加法同态性

Pedersen 承诺具备加法同态特性，这是椭圆曲线点运算的性质决定的。

假设有两个要承诺的信息 v_1 、 v_2 ，随机数 r_1 、 r_2 ，生成对应的两个承诺为：

$$C(v_1) = r_1G + v_1H, C(v_2) = r_2G + v_2H$$

显然，满足加法同态特性：

$$C(v_1) + C(v_2) = (r_1G + v_1H) + (r_2G + v_2H) = (r_1G + v_1H) + (r_2G + v_2H) = C(v_1 + v_2)。$$

3.1.4 应用示例

Pedersen 承诺有很多用途，

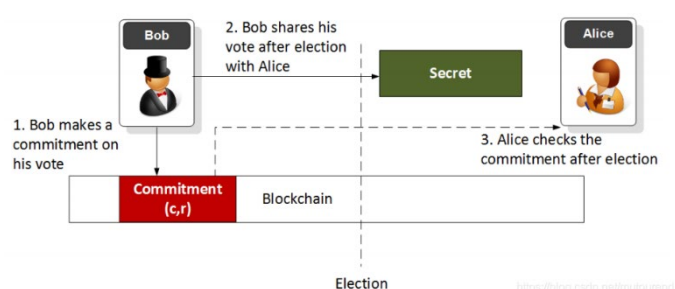
允许我们承诺一个信息，但在未来的某个时候才真正透露出来。我们还可以使用 Pedersen 承诺来实现一些不解密隐私信息时候的验证。

(1) 信息隐私性保护

Pedersen 承诺允许我们承诺一个秘密信息，在未来的某个时候才真正透露出来。

(2) 电子投票

Pedersen Commitment: Voting



因为承诺具有绑定性，所承诺的秘密信息是不允许更改的，在未来透露出来的时候可以进行验证。因此，Pedersen 承诺可以用于电子投票时候对所投的人进行承诺和事后校验。

(3) 隐私交易

Pedersen 承诺还可以用来实现匿名保密交易的协议。

假定在区块链上，Alice 转给 Bob 一定数额的数字货币，在其他人不了解数额和地址的情况下，使用 Pedersen 承诺可以保证这笔交易是有效的，使得任何人在区块浏览器上都查不到数额和地址信息。如何在不知道交易金额的情况下验证这笔交易收支平衡呢？

具体点，我们假定 Alice 有 10 元钱，要转给 Bob 的额度为 7 元，基于 Pedersen 承诺来实现的具体步骤如图所示。

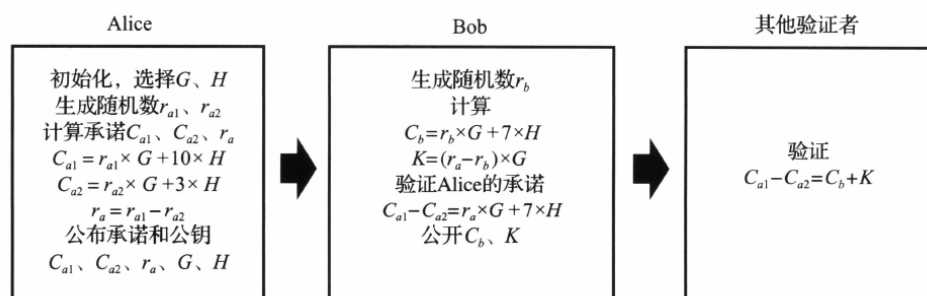


图 采用 Pedersen 同态承诺验证转账示意图

Alice 对参与交易的 10 元和余额 3 元都做了承诺，即 C_{a1} 和 C_{a2} ，并公布了两个关联的随机数的差值 r_a 。Bob 对收到的金额 7 元钱也做了承诺，且可以通过 C_{a1} 和 C_{a2} 来验证 Alice 的承诺是正确的。

其他验证者可以通过 Alice 和 Bob 公开的承诺和相关信息进行整个交易的验证。显然，验证过程不会泄露双方的余额，也不会泄露交易的金额。即使攻击者可以通过计算 $K = C_{a1} - C_{a2} - C_b$ 得到 $7 \cdot H$ ，但是，因为椭圆曲线离散对数求解困难问题，其无法反推出 7。同理，其他验证者也无法推断出 Alice 原有金额以及找零金额。

虽然 Pederson 承诺证明了数字之间的关系，但是并没有限制任何数字的取值区间，如果提供的承诺出现了负数，则违背了余额不能为负的现实约束，使得整个数字货币体系不可用。因此，还需要对隐藏的数值进行范围证明。不允许泄露隐私情况下证明数值的范围，就得依赖零知识证明了。

3.2 零知识证明

3.2.1 基本概念

1. 定义

零知识（Zero-Knowledge, ZK）证明允许证明方让验证方相信证明方自己知道一个满足 $C(x)=1$ 的 x ，但不会进一步泄漏关于 x 的任何信息。这里 C 是一个公开的谓词函数。

零知识证明是由 S.Goldwasser、S.Micali 及 C.Rackoff 在 20 世纪 80 年代初提出的，是一种涉及两方或更多方的协议，允许证明者能够在不向验证者提供任何有用的信息的情况下，使验证者相信某个论断是正确的。

举一个简单的例子，有一个缺口环形的长廊，出口和入口距离非常近（在目距之内），但走廊中间某处有一道只能用钥匙打开的门，A 要向 B 证明自己拥有该门的钥匙。采用零知识证明，则 B 看着 A 从入口进入走廊，然后又从出口走出走廊，这时 B 没有得到任何关于这个钥匙的信息，但是完全可以证明 A 拥有钥匙。

2. 性质

在零知识证明中，需要满足三个性质：

正确性：没有人能够假冒证明方 P 使这个证明成功。如果不满足这条性质，也就是证明方 P 不知道“知识”，再怎么证明，验证方 V 也很难相信证明方 P 拥有正确的知识。

完备性：如果证明方 P 和验证方 V 都是诚实的，并证明过程的每一步都进行正确的计算，那么这个证明一定是成功的。也就是说如果证明方 P 知道“知识”，那么验证方 V 会有极大的概率相信证明方 P。

零知识性：证明执行完之后，验证方 V 只获得了“证明方 P 拥有这个知识”这条信息，而没有获得关于这个知识本身的任何信息。

3. 应用

零知识证明的应用场景很多，简要举例如下。

身份认证：可用于对用户进行身份验证，而无需交换密码等机密信息。比如，用户可以向网站证明，他拥有私钥，网站并不需要知道私钥的内容，可以通过验证这个零知识证明，确认用户的身份。

去中心化存储：服务器可以向用户证明他们的数据被妥善保存并且不泄露数据的内容。

数据的隐私保护：在隐私场景中，根据零知识性，不泄漏交易的接收方、发送方、交易细节的前提下，满足特定业务的需求。比如：

- ✧ 买保险的时候，保险公司需要了解是否患有某种疾病，但是我不想让保险公司知道我的全部病历信息，那我可以证明给保险公司看，我没有相关疾病就足够了；
- ✧ 在金融领域，抵押贷款申请人可以证明他们的收入在可接受的范围内，而不透露他们的确切工资；
- ✧ 在区块链上，比特币和以太坊等区块链能保证链上数据的透明性使人人人都可以验证链上交易，这意味着参与者几乎没有了隐私，可能导致数据的非对称性，而零知识证明可以帮助保护区块链参与者的隐私权。

3.2.2 交互式 Schnorr 协议

Schnorr 机制是一种基于离散对数难题的零知识证明机制。证明者声称知道一个密钥 sk 的值，通过使用 Schnorr 加密技术，可以在不揭露 sk 的情况下，向验证者证明对 sk 的知情权。可用于证明你有一个私钥但是不披露私钥的内容。这是一类典型的可以应用到前面所述的身份认证场景的零知识证明协议。

依据椭圆曲线的离线对数难题，我们知道已知椭圆曲线 E 和生成元 G ，随机选择一个整数 d ，容易计算 $Q=d*G$ ，但是给定的 Q 和 G 计算 d 就非常困难。假设 Alice 拥有一个秘密数字 a ，我们可以把这个数字当成私钥 sk ，然后把它映射到椭圆曲线群上的一个点 $a*G$ ，简写为 aG 。这个点我们把它当做公钥 PK 。接下来，Alice 向 Bob 证明她拥有 PK 对应的私钥 sk ，那么如何证明呢？

（1）协议流程

承诺阶段：Alice 产生一个随机数 r ，计算 $R=r*G$ 并发送 R 给 Bob。

挑战阶段：Bob 提供一个随机数 c 进行挑战，并将 c 发送给 Alice。

回应挑战：Alice 根据挑战数 c 计算 $z=r+a*c$ ，然后把 z 发给 Bob。

验证阶段：Bob 通过式子进行检验： $z*G ?= R + c*PK$ 。

由于 $z=r+c*sk$ ，所以 $z*G=rG+c*(aG)=R+c*PK$ 。如果相等则证明 Alice 确实拥有私钥 sk ，但是验证者 Bob 并不能得到私钥 sk 的值，因此这个过程是零知识的。

(2) 安全性

由于椭圆曲线上的离散对数问题，知道 R 和 G 的情况下通过 $R=r*G$ 解出 r 是不可能的，所以保证了 r 的私密性。但是，整个过程是在证明者和验证者在私有安全通道中执行的。这是由于协议存在交互过程，只对参与交互的验证者有效，其他不参与交互的验证者，无法判断整个过程是否存在串通的舞弊行为。

该协议如果用在非交互情况下会存在一些问题：

- ✧ 问题一：私钥泄露。一旦两个验证者相互串通，交换自己得到的值，便可以推出私钥。因此，是无法公开验证的。不妨来个数学理论推导：在公开验证的条件下，两个验证者分别提供两个不同的随机值 c_1 和 c_2 ，并要求证明者计算 $z_1 = r + c_1 * sk$ ， $z_2 = r + c_2 * sk$ ，即可计算出 $sk = (z_1 - z_2) / (c_1 - c_2)$ 。因此，这个过程便无法公开验证。
- ✧ 问题二：造假。为什么需要验证者回复一个随机数 c 呢？这是为了防止 Alice 造假。如果 Bob 不回复一个 c ，就变成一次性交互。因为 a 和 r 都是 Alice 自己生成的，她知道 Bob 会用 PK 和 R 相加然后再与 $z * G$ 进行比较。所以她完全可以在不知道 a 的情况下构造： $R = r * G - PK$ 和 $z = r$ 。这样 Bob 的验证过程就变成： $z * G \stackrel{?}{=} PK + R \implies r * G \stackrel{?}{=} PK + r * G - PK$ 。这是永远成立的，所以这种方案并不正确。

3.2.3 非交互式零知识证明

上述 Schnorr 协议中存在的私钥泄露问题使得算法无法在公开的环境下使用。

通过将原始的交互式协议转变为非交互式协议可以解决这个问题。

(1) 非交互式 Schnorr 协议

承诺阶段：Alice 均匀随机选择 r ，并依次计算 $R=r*G$, $c=\text{Hash}(R,PK)$, $z=r+c*sk$ ，然后生成证明 (R,z) 。

验证阶段：Bob(或者任意一个验证者)计算 $c=\text{Hash}(PK,R)$ ，验证 $z*G \stackrel{?}{=} R+c*PK$ 。

安全性。为了不让 Alice 进行造假，在交互式 Schnorr 协议中需要 Bob 发送一个 c 值，并将 c 值构造进公式中。所以，在非交互式 Schnorr 协议中，如果 Alice 选择一个无法造假并且大家公认的 c 值并将其构造进公式中，问题就解决了。生成这个公认无法造假的 c 的方法是使用随机数预言机。

随机数预言机（见 1.4.5 节）是一种针对任意输入得到的输出是独立且均匀分布的哈希函数。理想的随机数预言机并不存在，在实现中，经常采用密码学哈希函数作为随机数预言机。一个密码学安全哈希函数是单向的，因此，虽然 c 是 Alice 计算的，但是 Alice 并没有能力实现通过挑选 c 来作弊。因为只要 Alice 一旦产生 R ， c 就相当于固定下来了。这样，就把三步 Schnorr 协议合并为一步。Alice 可直接发送 (R,z) ，因为 Bob 拥有 Alice 的公钥 PK ，于是 Bob 可自行计算出 c 。然后验证 $z*G \stackrel{?}{=} c*PK+R$ 。

（2）典型非交互零知识证明设计思想

交互式零知识证明需要证明者和验证者时刻保持在线状态，而这会因网络延迟、拒绝服务等原因难以保障。而在非交互式零知识证明(Non-Interactive Zero Knowledge, NIZK)中，证明者仅需发送一轮消息即可完成证明。然而，在标准假设下已证明无法构造针对非平凡语言的非交互式零知识证明系统（平凡问题虽然也可以零知识证明，但没有意义，因为验证者直接可以在多项式时间内根据输出求解出秘密输入）。因此必须引入新的假设。

目前主流的非交互式零知识证明的构造方法有两种，一是基于随机预言机并利用 Fiat-Shamir 启发式实现的，二是基于 CRS (Common Reference String)模型实现的。

上述非交互式 Schnorr 协议就是基于 ROM 并利用 Fiat-Shamir 变换实现的非交互式零知识证明协议。Fiat-Shamir 变换，又叫 Fiat-Shamir Heuristic（启发式），或者 Fiat-Shamir Paradigm（范式），由 Fiat 和 Shamir 在 1986 年提出，其特点是可以将交互式零知识证明转换为非交互式零知识证明，思路就是用公开的哈希函数的输出代替随机的挑战。

除了采用随机预言机之外，非交互零知识证明系统也可以采用公共参考串 CRS 完成随机挑战。它是在证明者构造证明之前由一个受信任的第三方产生的随机字符串，CRS 必须由一个受信任的第三方来完成，同时共享给证明者和验证者。CRS 其实就把挑战过程中所要生成的随机数和挑战数，都预先生成好，然后基于这些随机数和挑战数生成他们对应的在证明和验证过程中所需用到的各种同态隐藏。之后，就把这些随机数和挑战数销毁。这些随机数和挑战数被称为 toxic waste（有毒废物）。之所以把他们称为 toxic waste，是因为如果他们没有被销毁的话，就可以伪造证明了。

（3）简洁的零知识证明 zkSNARK

zkSNARK(zero-knowledge Succinct Non-interactive Arguments of Knowledge)就是一类基于 CRS 模型实现的典型的非交互式零知识证明技术。zkSNARK 中比较典型的协议有 Groth10、GGPR13、Pinocchio、GRoth6、GKMMM18 等。

zkSNARK 的命名几乎包含其所有技术特征：

- 简洁性：最终生成的证明具有简洁性，也就是说最终生成的证明足够小，并且与计算量大小无关。
- 无交互：没有或者只有很少的交互。对于 zkSNARK 来说，证明者向验证者发送一条信息之后几乎没有交互。此外，zkSNARK 还常常拥有“公共验证者”的属性，意思是在没有再次交互的情况下任何人都可以验证。
- 可靠性：证明者在不知道见证（Witness，私密的数据，只有证明者知道）的情况下，构造出证明是不可能的（这样的证明系统叫作一个 Argument）。
- 零知识：验证者无法获取证明者的任何隐私信息。

应用 zkSNARK 技术实现一个非交互式零知识证明应用的开发步骤大体如下：

- 定义电路：将所要声明的内容的计算算法用算术电路来表示，简单地说，算术电路以变量或数字作为输入，并且允许使用加法、乘法两种运算来操作表达式。所有的

NP 问题都可以有效地转换为算术电路。

- 将电路表达为 R1CS: 在电路的基础上构造约束, 也就是 R1CS (Rank- Constraint System, 一阶约束系统), 有了约束就可以把 NP 问题抽象成 QAP (Quadratic Arithmetic Problem) 问题。R1CS 与 QAP 形式上的区别是 QAP 使用多项式来代替点积运算, 而它们的实现逻辑完全相同。有了 QAP 问题的描述, 就可以构建 zkSNARKs。
- 完成应用开发:
 - ✧ 生成密钥: 生成证明密钥 (Proving Key) 和验证密钥 (Verification Key);
 - ✧ 生成证明: 证明方使用证明密钥和其可行解构造证明;
 - ✧ 验证证明: 验证方使用验证密钥验证证明方发过来的证明。

基于 zkSNARK 的实际应用, 最终实现的效果就是证明者给验证者一段简短的证明, 验证者可以自行校验某命题是否成立。

3.2.4 应用示例

1. libsnark 环境搭建

libsnark 是用于开发 zkSNARK 应用的 C++代码库, 由 SCIPR Lab 开发并采用商业友好的 MIT 许可证 (但附有例外条款) 在 GitHub 上 (<https://github.com/scipr-lab/libsnark>) 开源。libsnark 框架提供了多个通用证明系统的实现, 其中使用较多的是 BCTV14a 和 Groth16。查看 libsnark/libsnark/zk_proof_systems 路径, 就能发现 libsnark 对各种证明系统的具体实现, 并且均按不同类别进行了分类, 还附上了实现依照的具体论文。其中:

- zk_proof_systems/ppzksnark/r1cs_ppzksnark 对应的是 BCTV14a
- zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark 对应的是 Groth16

ppzksnark 是指 preprocessing zkSNARK。这里的 preprocessing 是指可信设置(trusted setup), 即在证明生成和验证之前, 需要通过一个生成算法来创建相关的公共参数 (证明密钥和验证密钥)。这个提前生成的参数就是公共参考串 CRS。

使用 libsnark 框架需要有简单的 C++语言基础, libsnark 源码目录如下:

— depends/	相关外部依赖
— tinyram_examples/	包含TinyRAM (一种比R1CS更高阶的描述问题的语言) 的例子
— libsnark/	主题代码目录
— common/	定义和实现了一些通用的数据结构, 例如默克尔树、稀疏向量等
— gadgetlib1/	抽象出一层以便构建R1CS (基于模板), 提供较丰富的gadget
— gadgetlib2/	接口不基于模板, 文档更好、更易用, 但gadget较少
— knowledge_commitment/	在 multiexp的基础上, 引入pair概念
— reductions	各种不同描述语言之间的转化
— relations	各种NPC问题描述语言的接口 (包括R1CS、USCS等)
— zk_proof_systems	各种证明系统 (包括Groth16、GM17等)

Libsnark 安装相对麻烦，它的多个子模块也需要编译安装。

1) 创建名为 Libsnark 的文件夹

2) 打开 https://github.com/sec-bit/libsnark_abc，点击“Code”、“Download ZIP”，下载后解压到 Libsnark 文件夹，得到~/Libsnark/libsnark_abc-master

3) 打开 <https://github.com/scipr-lab/libsnark>，点击“Code”、“Download ZIP”，下载解压后，将其中文件复制到~/Libsnark/libsnark_abc-master/depends/libsnark 文件夹内

4) 打开 <https://github.com/scipr-lab/libsnark>，点击“depends”，可以看到六个子模块的链接地址

 ate-pairing @ e698901	Update git submodules to latest commits on origin	6 years ago
 gtest @ 3a4cf1a	Update libqfft, libff, gtest, xbyak dependency versions	6 years ago
 libff @ 176f3f4	Updated libff, libqfft submodules to their latest staging branch	3 years ago
 libqfft @ 7e1e957	Updated libff, libqfft submodules to their latest staging branch	3 years ago
 libsnark-supercop @ b04a0ea	Update third_party to depends	6 years ago
 xbyak @ f0a8f7f	Update libqfft, libff, gtest, xbyak dependency versions	6 years ago

分别点击这六个链接并下载解压，得到如下六个文件夹，为方便下文表述，分别将这六个文件夹命名为 Libqfft、Libff、Gtest、Xbyak、Ate-pairing、Libsnark-supercop。

 libqfft-7e1e957d0e84accadc92e88162510c0ad88...	7 个项目	2020 年 3 月 30 日	☆
 libff-176f3f42fdef791f12b24417a400c4b6d386863c	6 个项目	2020 年 3 月 30 日	☆
 googletest-3a4cf1a02ef4adc28fccb7eef2b573b14...	12 个项目	2018 年 2 月 24 日	☆
 xbyak-f0a8f7faa27121f28186c2a7f4222a9fc66c283d	10 个项目	2018 年 2 月 14 日	☆
 ate-pairing-e69890125746cdf25b5b51227d96678...	14 个项目	2017 年 7 月 6 日	☆
 libsnark-supercop-b04a0ea2c7d7422d74a512ce84...	5 个项目	2015 年 6 月 4 日	☆

5) 选择对应的 Linux 系统，执行以下命令

Ubuntu 18.04 LTS, Ubuntu 20.04 LTS:

```
sudo apt install build-essential cmake git libgmp3-dev libprocps-dev python3-markdown libboost-program-options-dev libssl-dev python3 pkg-config
```

Ubuntu 16.04 LTS:

```
sudo apt-get install build-essential cmake git libgmp3-dev libprocps4-dev python-markdown libboost-all-dev libssl-dev
```

Ubuntu 14.04 LTS:

```
sudo apt-get install build-essential cmake git libgmp3-dev libprocps4-dev python-markdown libboost-all-dev libssl-dev
```

6) 安装子模块 xbyak

将下载得到的文件夹 Xbyak 内的文件复制到~/Libsnark/libsnark_abc-master/depends/libsnark/depends/xbyak，并在该目录下打开终端，执行以下命令

```
sudo make install
```

出现以下代码，说明安装成功

```
cpz2000@cpz2000-virtual-machine:~/Libsnark/libsnark_abc-master/depends/libsnark/depends/xbyak$ sudo make install
[sudo] cpz2000 的密码:
mkdir -p /usr/local/include/xbyak
cp -pR xbyak/*.h /usr/local/include/xbyak
```

7) 安装子模块 ate-pairing

将下载得到的文件夹 Ate-pairing 内的文件复制到~/Libsnark/libsnark_abc-master/depends/libsnark/depends/ate-pairing，并在该目录下打开终端，执行以下命令

```
make -j
test/bn
```

执行 make -j 最后几行如下：

```
g++ -o loop_test loop_test.o -lm -lzm -L../lib -lgmp -lgmpxx -m64
g++ -o java_api java_api.o -lm -lzm -L../lib -lgmp -lgmpxx -m64
g++ -o sample sample.o -lm -lzm -L../lib -lgmp -lgmpxx -m64
g++ -o bench_test bench.o -lm -lzm -L../lib -lgmp -lgmpxx -m64
g++ -o test_zm test_zm.o -lm -lzm -L../lib -lgmp -lgmpxx -m64
g++ -o bn bn.o -lm -lzm -L../lib -lgmp -lgmpxx -m64
make[1]: 离开目录“/home/cpz2000/Libsnark/libsnark_abc-master/depends/libsnark/depends/ate-pairing/test”
```

执行 test/bn 最后几行如下：

```
Fp2::mul      253.01 clk
Fp2::inverse  18.372Kclk
Fp2::square   201.28 clk
Fp2::mul_xi    19.73 clk
Fp2::mul_Fp_0 196.64 clk
Fp2::mul_Fp_1 196.28 clk
Fp2::divBy2    71.63 clk
Fp2::divBy4    94.46 clk
finalexpr 434.492Kclk
pairing 898.251Kclk
precomp 159.103Kclk
millerLoop 429.461Kclk
Fp::add 11.18 clk
Fp::sub 11.98 clk
Fp::neg 7.49 clk
Fp::mul 94.65 clk
Fp::inv 9.869Kclk
mul256 33.39 clk
mod512 61.48 clk
Fp::divBy2 24.85 clk
Fp::divBy4 52.13 clk
err=0(test=461)
cpz2000@cpz2000-virtual-machine:~/Libsnark/libsnark_abc-master/depends/libsnark/depends/ate-pairing$
```

8) 安装子模块 libsnark-supercop

将下载得到的文件夹 Libsnark-supercop 内的文件复制到~/Libsnark/libsnark_abc-master/depends/libsnark/depends/libsnark-supercop，并在该目录下打开终端，执行以下命令

```
./do
```

出现以下代码，说明安装成功

```
cpz2000@cpz2000-virtual-machine:~/Libsnark/libsnark_abc-master/depends/libsnark/depends/libsnark-supercop$ ./do
ar: 正在创建 ../lib/libsupercop.a
```

9) 安装子模块 gtest

将下载得到的文件夹 Gtest 内的文件复制到~/Libsnark/libsnark_abc-master/depends/libsnark/depends/gtest

10) 安装子模块 libff

将下载得到的文件夹 Libff 内的文件复制到~/Libsnark/libsnark_abc-master/depends/libsnark/depends/libff。点击 libff->depends，可以看到一个 ate-pairing 文件夹和一个 xbyak 文件夹，这是 libff 需要的依赖项。打开这两个文件夹，会发现它们是空的，这时候需要将下载得到的 Ate-pairing 和 Xbyak 内的文件复制到这两个文件夹下。

在~/Libsnark/libsnark_abc-master/depends/libsnark/depends/libff 下打开终端，执行命令：

```
mkdir build
cd build
cmake ..
make
sudo make install
```

安装完之后检测是否安装成功，执行以下命令

```
make check
```

如下图所示则安装成功：

```
[ 90%] Built target algebra_fields_test
[ 92%] Building CXX object libff/CMakeFiles/algebra_bilinearity_test.dir/algebra
/curves/tests/test_bilinearity.cpp.o
[ 95%] Linking CXX executable algebra_bilinearity_test
[ 95%] Built target algebra_bilinearity_test
[ 97%] Building CXX object libff/CMakeFiles/algebra_groups_test.dir/algebra/curv
es/tests/test_groups.cpp.o
[100%] Linking CXX executable algebra_groups_test
[100%] Built target algebra_groups_test
Test project /home/cpz2000/Libsnark/libsnark_abc-master/depends/libsnark/depends
/libff/build
  Start 1: algebra_bilinearity_test
1/3 Test #1: algebra_bilinearity_test ..... Passed    0.00 sec
  Start 2: algebra_groups_test
2/3 Test #2: algebra_groups_test ..... Passed    0.00 sec
  Start 3: algebra_fields_test
3/3 Test #3: algebra_fields_test ..... Passed    0.00 sec

100% tests passed, 0 tests failed out of 3

Total Test time (real) =  0.01 sec
[100%] Built target check
cpz2000@cpz2000-virtual-machine:~/Libsnark/libsnark_abc-master/depends/libsnark/
depends/libff/build$
```

11) 安装子模块 libfqfft

将下载得到的文件夹 Libfqfft 内的文件复制到~/Libsnark/libsnark_abc-master/depends/libsnark/depends/libfqfft。点击 libfqfft->depends，可以看到 libfqfft 有四个依赖项，分别是 ate-pairing、gtest、libff、xbyak，点开来依然是空的。和上一步一样，将下载得到的文件夹内文件复制到对应文件夹下。注意 libff 里还有 depends 文件夹，里面的 ate-pairing 和 xbyaky 也是空的，需要将下载得到的 airing 和 Xbyak 文件夹内的文件复制进去。

在~/Libsnark/libsnark_abc-master/depends/libsnark/depends/libfqqft 下打开终端，执行命令：

```
mkdir build
cd build
cmake ..
make
sudo make install
```

安装完之后检测是否安装成功，执行以下命令

```
make check
```

如下图所示则安装成功

```
evaluation_domain_test.cpp.o
[ 93%] Linking CXX executable evaluation_domain_test
[ 93%] Built target evaluation_domain_test
[ 95%] Building CXX object libfqqft/CMakeFiles/polynomial_arithmetic_test.dir/te
sts/init_test.cpp.o
[ 97%] Building CXX object libfqqft/CMakeFiles/polynomial_arithmetic_test.dir/te
sts/polynomial_arithmetic_test.cpp.o
[100%] Linking CXX executable polynomial_arithmetic_test
[100%] Built target polynomial_arithmetic_test
Test project /home/cpz2000/Libsnark/libsnark_abc-master/depends/libsnark/depends
/libfqqft/build
  Start 1: evaluation_domain_test
1/3 Test #1: evaluation_domain_test ..... Passed    0.00 sec
  Start 2: polynomial_arithmetic_test
2/3 Test #2: polynomial_arithmetic_test ..... Passed    0.00 sec
  Start 3: kronecker_substitution_test
3/3 Test #3: kronecker_substitution_test ..... Passed    0.00 sec

100% tests passed, 0 tests failed out of 3

Total Test time (real) =  0.01 sec
[100%] Built target check
cpz2000@cpz2000-virtual-machine:~/Libsnark/libsnark_abc-master/depends/libsnark/
depends/libfqqft/build$
```

12) libsnark 编译安装

在~/Libsnark/libsnark_abc-master/depends/libsnark 下打开终端，执行以下命令：

```
mkdir build
cd build
cmake ..
make
make check
```

如下图所示则安装成功


```

18/23 Test #18: zk_proof_systems_tbcspzksnark_test ..... Passed 0
.00 sec
      Start 19: zk_proof_systems_uscsppzksnark_test
19/23 Test #19: zk_proof_systems_uscsppzksnark_test ..... Passed 0
.00 sec
      Start 20: test_knapsack_gadget
20/23 Test #20: test_knapsack_gadget ..... Passed 0
.00 sec
      Start 21: test_merkle_tree_gadgets
21/23 Test #21: test_merkle_tree_gadgets ..... Passed 0
.01 sec
      Start 22: test_set_commitment_gadget
22/23 Test #22: test_set_commitment_gadget ..... Passed 8
.46 sec
      Start 23: test_sha256_gadget
23/23 Test #23: test_sha256_gadget ..... Passed 0
.05 sec

100% tests passed, 0 tests failed out of 23

Total Test time (real) = 55.55 sec
[100%] Built target check
cpz2000@cpz2000-virtual-machine:~/Libsnark/libsnark_abc-master/depends/libsnark/
build$

```

13) 整体编译安装

在~/Libsnark/libsnark_abc-master 下打开终端，执行以下命令：

```

mkdir build

cd build

cmake ..

make

```

14) 运行代码

执行以下命令：

```
./src/test
```

最终出现如下日志，则说明你已顺利拥有了 zkSNARK 应用开发环境，并成功跑了第一个 zk-SNARKs 的 demo。

```

      (leave) Call to alt_bn128_final_exponentiation [0.0036s x1.00](
1676981610.8501s x0.00 from start)
      (leave) Check QAP divisibility [0.0075s x1.00] (1676981
610.8501s x0.00 from start)
      (leave) Online pairing computations [0.0076s x1.00] (1676981
610.8501s x0.00 from start)
      (leave) Call to r1cs_gg_ppzksnark_online_verifier_weak_IC [0.0076s x1.00](
1676981610.8501s x0.00 from start)
      (leave) Call to r1cs_gg_ppzksnark_online_verifier_strong_IC [0.0076s x1.00](
1676981610.8501s x0.00 from start)
      (leave) Call to r1cs_gg_ppzksnark_verifier_strong_IC [0.0088s x1.00] (1676981
610.8501s x0.00 from start)
Number of R1CS constraints: 4
Primary (public) input: 1
35

Auxiliary (private) input: 4
3
9
27
30

Verification status: 1
cpz2000@cpz2000-virtual-machine:~/Libsnark/libsnark_abc-master/build$

```

3. 应用示例

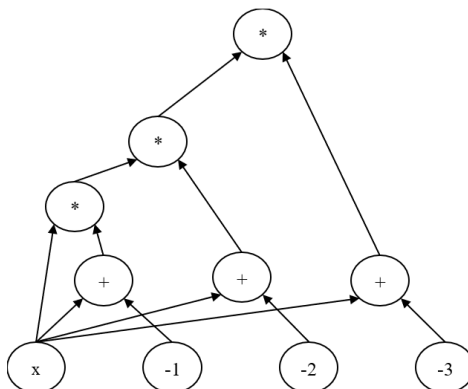
实验 3.1 假设证明方有一个整数 x ，他希望向验证方证明这个整数 x 的取值范围为 $[0,3]$ 。

(1) 将待证明的命题表达为 RICS

1) 用算术电路表示待证明问题

在计算复杂性理论中，计算多项式的最自然计算模型就是算术电路。简单地说，算术电路以变量或数字作为输入，并且允许使用加法、乘法两种运算来操作表达式。

如果想要约束整数 x 必须取值于 $[a, b]$ 的话，可以用如下公式来约束它： $(x - a) * (x - (a + 1)) * \dots * (x - b) = 0$ 。对于待证明的问题证明整数 x 的取值范围为 $[0,3]$ ，可以写出如下约束： $x(x - 1)(x - 2)(x - 3) = 0$ ，该约束对应的算术电路如下图所示。



可以用 $C(v, w)$ 来表示上述电路，其中 v 为公有输入，表达了想要证明的问题的特性和一些固定的环境变量，所有人都知道 v ； w 为私密输入，只有证明方才会知道。

将待证明命题转换为算术电路之后，就可以将证明过程转换为证明 $C(v, w) = 0$ ，即在证明方和验证方已知算术电路 C 的输出为 0，并且公有输入为 v 的情况下，证明方需要证明他拥有能构成电路输出为 0 的私密输入值 w 。

2) 用 RICS 描述电路

首先，将算数电路拍平成多个 $x = y$ 或者 $x = y (op) z$ 形式的等式，其中 op 可以是加、减、乘、除运算符中的一种。对于 $x(x - 1)(x - 2)(x - 3) = 0$ ，可以拍平成如下几个等式：

$$w_1 = x - 1$$

$$w_2 = x - 2$$

$$w_3 = x - 3$$

$$w_4 = x * w_1$$

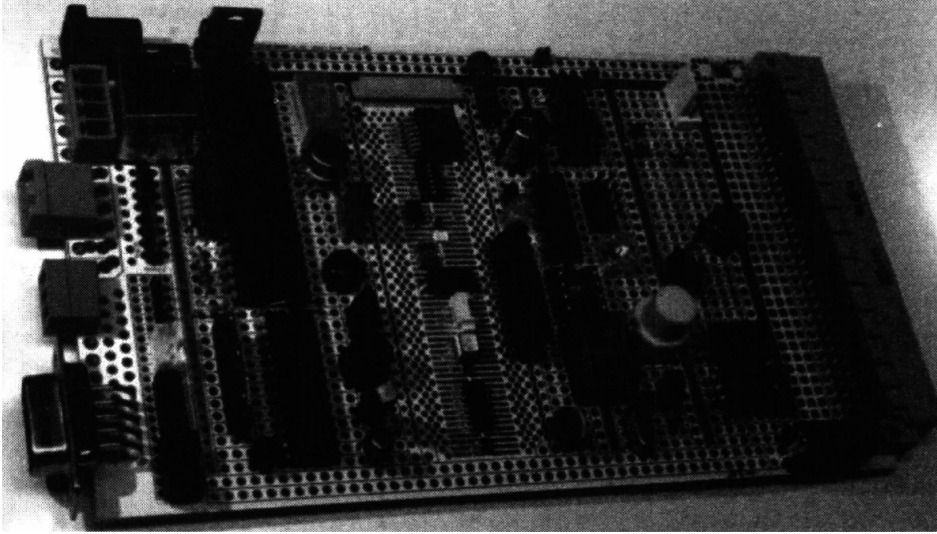
$$w_5 = w_2 * w_4$$

$$out = w_3 * w_5$$

3) 使用原型板搭建电路

然后，使用原型板 protoboard 搭建电路。

在电气工程中，原型板（Protoboard）是用于连接电路和芯片的，如图所示。



将待证明的命题用电路表示，并用 R1CS 描述电路之后，就可以构建一个 protoboard。protoboard，也就是原型板或者面包板，可以用来快速搭建算术电路，把所有变量、组件和约束关联起来。

因为在初始设置、证明、验证三个阶段都需要构造面包板，所以这里将下面的代码放在一个公用的文件 common.hpp 中供三个阶段使用。

```
//代码开头引用了三个头文件：第一个头文件是为了引入 default_r1cs_gg_ppzksnark_pp 类型；第二个则为了引入证明相关的各个接口；pb_variable 则是用来定义电路相关的变量。

#include <libsark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>

#include <libsark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>

#include <libsark/gadgetlib1/pb_variable.hpp>

using namespace libsark;

using namespace std;

//定义使用的有限域

typedef libff::Fr<default_r1cs_gg_ppzksnark_pp> FieldT;

//定义创建面包板的函数

protoboard<FieldT> build_protoboard(int* secret)

{

    //初始化曲线参数

    default_r1cs_gg_ppzksnark_pp::init_public_params();

    //创建面包板

    protoboard<FieldT> pb;
```



```
//定义所有需要外部输入的变量以及中间变量
```

```
pb_variable<FieldT> x;  
pb_variable<FieldT> w_1;  
pb_variable<FieldT> w_2;  
pb_variable<FieldT> w_3;  
pb_variable<FieldT> w_4;  
pb_variable<FieldT> w_5;  
pb_variable<FieldT> out;
```

//下面将各个变量与 **protoboard** 连接，相当于把各个元器件插到“面包板”上。
allocate()函数的第二个 **string** 类型变量仅是用来方便 **DEBUG** 时的注释，方便 **DEBUG** 时查看日志。

```
out.allocate(pb, "out");  
x.allocate(pb, "x");  
w_1.allocate(pb, "w_1");  
w_2.allocate(pb, "w_2");  
w_3.allocate(pb, "w_3");  
w_4.allocate(pb, "w_4");  
w_5.allocate(pb, "w_5");
```

//定义公有的变量的数量，**set_input_sizes(n)**用来声明与 **protoboard** 连接的 **public** 变量的个数 **n**。在这里 **n=1**，表明与 **pb** 连接的前 **n = 1** 个变量是 **public** 的，其余都是 **private** 的。因此，要将 **public** 的变量先与 **pb** 连接（前面 **out** 是公开的）。

```
pb.set_input_sizes(1);  
  
//为公有变量赋值  
pb.val(out)=0;
```

//至此，所有变量都已经顺利与 **protoboard** 相连，下面需要确定的是这些变量间的约束关系。如下调用 **protoboard** 的 **add_r1cs_constraint()**函数，为 **pb** 添加形如 $a * b = c$ 的 **r1cs_constraint**。即 **r1cs_constraint<FieldT>(a, b, c)**中参数应该满足 $a * b = c$ 。根据注释不难理解每个等式和约束之间的关系。

```
// x-1= w_1  
pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x-1, 1, w_1));  
  
// x-2= w_2  
pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x-2, 1, w_2));  
  
// x-3= w_3  
pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x-3, 1, w_3));  
  
// x*w_1=w_4  
pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x, w_1, w_4));
```

```

//w_2*w_4=w_5
pb.add_r1cs_constraint(r1cs_constraint<FieldT>(w_2, w_4, w_5));

//w_3*w_5=out
pb.add_r1cs_constraint(r1cs_constraint<FieldT>(w_3, w_5, out));

//证明者在生成证明阶段传入私密输入，为私密变量赋值，其他阶段为 NULL
if (secret!=NULL)
{
    pb.val(x)=secret[0];
    pb.val(w_1)=secret[1];
    pb.val(w_2)=secret[2];
    pb.val(w_3)=secret[3];
    pb.val(w_4)=secret[4];
    pb.val(w_5)=secret[5];
}

return pb;
}

```

（2）生成证明密钥和验证密钥

至此，针对命题的电路已构建完毕。

接下来，是生成公钥的初始设置阶段（Trusted Setup）。在这个阶段，我们把生成的证明密钥和验证密钥输出到对应文件中保存。其中，证明密钥供证明者使用，验证密钥供验证者使用。编写代码如下，将这段代码放在 `mysetup.cpp` 中。

```

#include <libsark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>

#include <libsark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>

#include <fstream>

#include "common.hpp"

using namespace libsark;
using namespace std;

int main()
{
    //构造面包板

```

```

    protoboard<FieldT> pb=build_protoboard(NULL);

    const r1cs_constraint_system<FieldT> constraint_system = pb.get_constraint_system();

    //生成证明密钥和验证密钥

    const r1cs_gg_ppzksnark_keypair<default_r1cs_gg_ppzksnark_pp> keypair = r1cs_gg_ppzksnark_generator<default_r1cs_gg_ppzksnark_pp>(constraint_system);

    //保存证明密钥到文件 pk.raw

    fstream pk("pk.raw",ios_base::out);

    pk<<keypair.pk;

    pk.close();

    //保存验证密钥到文件 vk.raw

    fstream vk("vk.raw",ios_base::out);

    vk<<keypair.vk;

    vk.close();


    return 0;

}

```

(3) 证明方使用证明密钥和其可行解构造证明

在定义面包板时，我们已为 public input 提供具体数值，在构造证明阶段，证明者只需为 private input 提供具体数值。再把 public input 以及 private input 的数值传给 prover 函数生成证明。生成的证明保存到 proof.raw 文件中供验证者使用。编写代码如下，将这段代码放在 myprove.cpp 中。

```

#include <libsark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>

#include <libsark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>

#include <fstream>

#include "common.hpp"

using namespace libsark;

using namespace std;

int main()
{
    //输入秘密值 x

    int x;

    cin>>x;

```

```

//为私密输入提供具体数值
int secret[6];
secret[0]=x;
secret[1]=x-1;
secret[2]=x-2;
secret[3]=x-3;
secret[4]=x*(x-1);
secret[5]=x*(x-1)*(x-2);
//构造面包板
protoboard<FieldT> pb=build_protoboard(secret);
const r1cs_constraint_system<FieldT> constraint_system = pb.get_constraint_system();
cout<<"公有输入: "<<pb.primary_input()<<endl;
cout<<"私密输入: "<<pb.auxiliary_input()<<endl;
//加载证明密钥
fstream f_pk("pk.raw",ios_base::in);
r1cs_gg_ppzksnark_proving_key<libff::default_ec_pp>pk;
f_pk>>pk;
f_pk.close();
//生成证明
const r1cs_gg_ppzksnark_proof<default_r1cs_gg_ppzksnark_pp> proof = r1cs_gg_ppzksnark_prover<default_r1cs_gg_ppzksnark_pp>(pk, pb.primary_input(), pb.auxiliary_input());
//将生成的证明保存到 proof.raw 文件
fstream pr("proof.raw",ios_base::out);
pr<<proof;
pr.close();
return 0;
}

```

(4) 验证方使用验证密钥验证证明方发过来的证明

最后我们使用 `verifier` 函数校验证明。如果 `verified = 1` 则说明证明验证成功。编写代码如下，将这段代码放在 `myverify.cpp` 中。

```

#include <libsark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>
#include <libsark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>

```

```

#include <fstream>
#include "common.hpp"
using namespace libsnark;
using namespace std;
int main()
{
    //构造面包板
    protoboard<FieldT> pb=build_protoboard(NULL);

    const r1cs_constraint_system<FieldT> constraint_system = pb.get_constraint_system();

    //加载验证密钥
    fstream f_vk("vk.raw",ios_base::in);
    r1cs_gg_ppzksnark_verification_key<libff::default_ec_pp>vk;
    f_vk>>vk;
    f_vk.close();

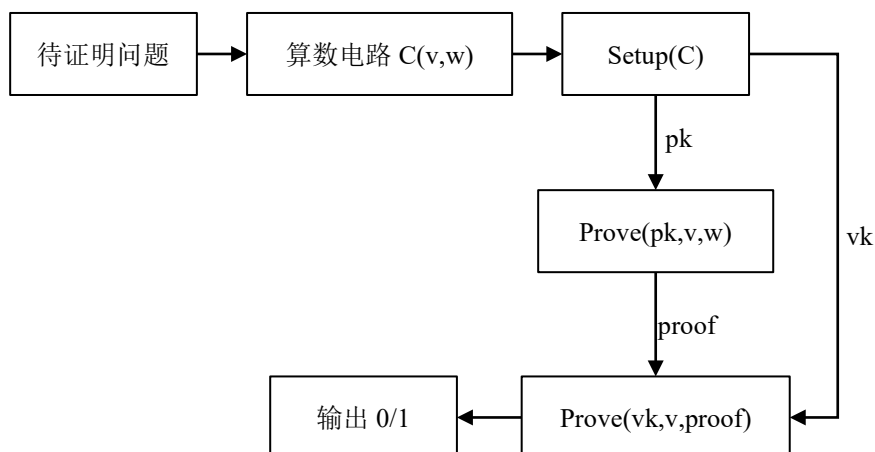
    //加载银行生成的证明
    fstream f_proof("proof.raw",ios_base::in);
    r1cs_gg_ppzksnark_proof<libff::default_ec_pp>proof;
    f_proof>>proof;
    f_proof.close();

    //进行验证
    bool verified = r1cs_gg_ppzksnark_verifier_strong_IC<default_r1cs_gg_ppzksnark_pp>(vk, pb.primary_input(), proof);

    cout<<"验证结果:"<<verified<<endl;

    return 0;
}

```



4. 编译运行

在~/Libsnark/libsnark_abc-master/src 下打开终端，输入如下命令，创建 common.hpp、mysetup.cpp、myprove.cpp、myverify.cpp 文件

```
touch common.hpp
touch mysetup.cpp
touch myprove.cpp
touch myverify.cpp
```

分别打开 common.hpp、mysetup.cpp、myprove.cpp、myverify.cpp，将对应代码复制进文件中。

打开~/Libsnark/libsnark_abc-master/src 目录下的 CMakeLists.txt 文件，将如下代码复制到文件末尾。

```
add_executable(
    mysetup

    mysetup.cpp
)
target_link_libraries(
    mysetup

    snark
)
target_include_directories(
    mysetup

    PUBLIC
    ${DEPENDS_DIR}/libsnark
    ${DEPENDS_DIR}/libsnark/depends/libfqfft
)

add_executable(
    myprove
```

```
    myprove.cpp
)
target_link_libraries(
    myprove

    snark
)
target_include_directories(
    myprove

    PUBLIC
    ${DEPENDS_DIR}/libsnaek
    ${DEPENDS_DIR}/libsnaek/depends/libfqfft
)

add_executable(
    myverify

    myverify.cpp
)
target_link_libraries(
    myverify

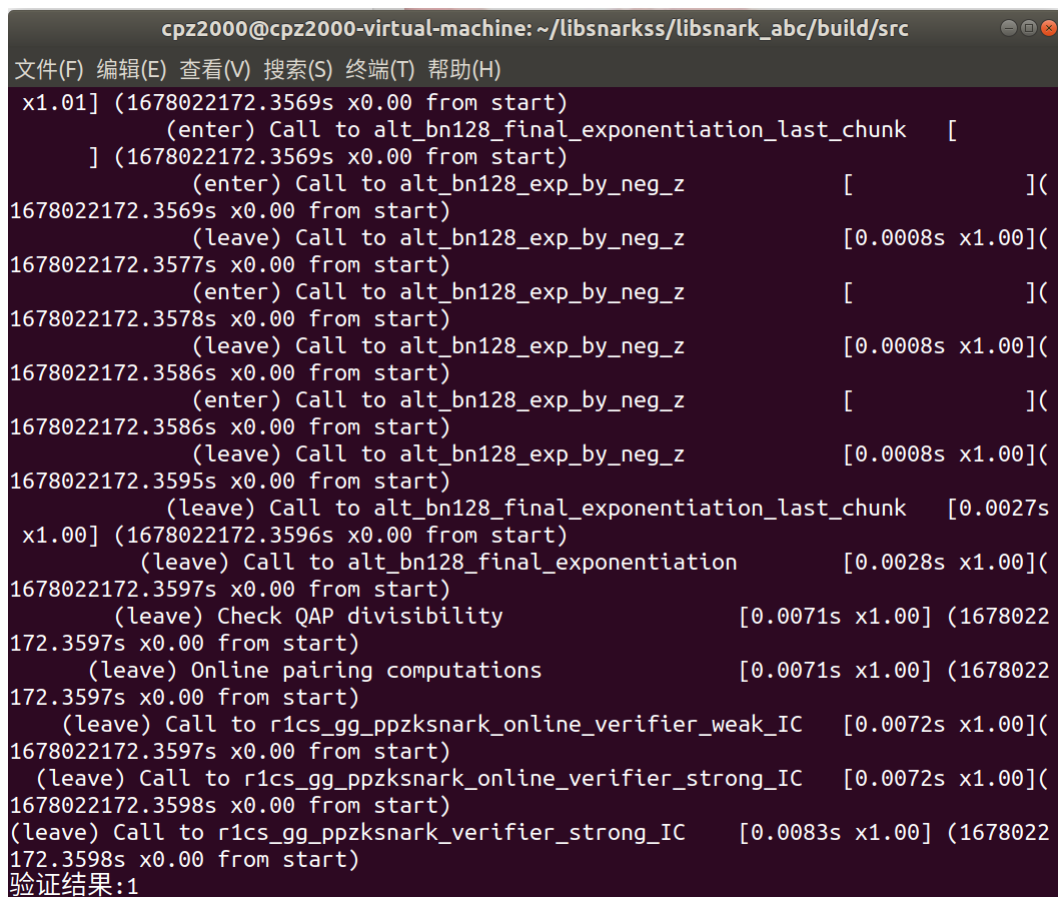
    snark
)
target_include_directories(
    myverify

    PUBLIC
    ${DEPENDS_DIR}/libsnaek
    ${DEPENDS_DIR}/libsnaek/depends/libfqfft
)
```


在~/Libsnark/libsnark_abc-master/build 下打开终端，执行以下命令：

```
cmake ..  
make  
cd src  
./mysetup  
./myprove  
2  
./myverify
```

运行结果如下：验证结果为 1，表示 $x = 2$ 在取值范围 $[0,3]$ 内。



```
cpz2000@cpz2000-virtual-machine: ~/libsarkss/libsnark_abc/build/src  
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)  
x1.01] (1678022172.3569s x0.00 from start)  
    (enter) Call to alt_bn128_final_exponentiation_last_chunk  [  
    ] (1678022172.3569s x0.00 from start)  
        (enter) Call to alt_bn128_exp_by_neg_z                [      ](  
1678022172.3569s x0.00 from start)  
        (leave) Call to alt_bn128_exp_by_neg_z                [0.0008s x1.00](  
1678022172.3577s x0.00 from start)  
        (enter) Call to alt_bn128_exp_by_neg_z                [      ](  
1678022172.3578s x0.00 from start)  
        (leave) Call to alt_bn128_exp_by_neg_z                [0.0008s x1.00](  
1678022172.3586s x0.00 from start)  
        (enter) Call to alt_bn128_exp_by_neg_z                [      ](  
1678022172.3586s x0.00 from start)  
        (leave) Call to alt_bn128_exp_by_neg_z                [0.0008s x1.00](  
1678022172.3595s x0.00 from start)  
    (leave) Call to alt_bn128_final_exponentiation_last_chunk  [0.0027s  
x1.00] (1678022172.3596s x0.00 from start)  
    (leave) Call to alt_bn128_final_exponentiation            [0.0028s x1.00](  
1678022172.3597s x0.00 from start)  
    (leave) Check QAP divisibility                            [0.0071s x1.00] (1678022  
172.3597s x0.00 from start)  
    (leave) Online pairing computations                        [0.0071s x1.00] (1678022  
172.3597s x0.00 from start)  
    (leave) Call to r1cs_gg_ppzksnark_online_verifier_weak_IC [0.0072s x1.00](  
1678022172.3597s x0.00 from start)  
    (leave) Call to r1cs_gg_ppzksnark_online_verifier_strong_IC [0.0072s x1.00](  
1678022172.3598s x0.00 from start)  
    (leave) Call to r1cs_gg_ppzksnark_verifier_strong_IC      [0.0083s x1.00] (1678022  
172.3598s x0.00 from start)  
验证结果:1
```

3.3 茫然传输

3.3.1 基本概念

茫然传输（Oblivious Transfer, OT）是安全多方计算协议的重要构造模块。2 选 1-OT 的标准定义涉及两个参与方：持有两个秘密值 x_0, x_1 的发送方 S ，持有一个选择比特 $b \in \{0, 1\}$ 的接收方 R 。OT 允许 R 得到 x_b ，但其无法得到与“另一个”秘密值 x_{1-b} 相关的任何信息。与此同时， S 无法得到任何信息。

举例，Alice 这儿有两个产品的折扣码。Bob 想获得其中一个，但又比较注重隐私，不想让 Alice 知道他选择了哪一个。这时候，他们就可以通过 OT 来完成交易。

OT 的形式化定义如下所述。

定义：2 选 1-OT 是一个密码学协议，它可以安全地实现图 2.1 所示的功能函数 F^{OT} 。

参数：

- 两个参与方：发送方 S 和接收方 R 。
- S 拥有两个秘密值 $x_0, x_1 \in \{0, 1\}^n$ ， R 拥有一个选择比特 $b \in \{0, 1\}$ 。

功能函数：

1. R 将 b 发送给 F^{OT} ， S 将 x_0, x_1 发送给 F^{OT} 。
2. F^{OT} 输出 x_b 给 R ，输出 $\perp$ 给 S 。

图 3.1 2 选 1-OT 功能函数 F^{OT}

还有很多 OT 变种协议，一种很容易想到的就是 k 选 1-OT，其中 S 拥有 k 个秘密值， R 拥有 $[0, \dots, k-1]$ 中的一个选择项。

3.3.2 基础构造

（1）基于公钥的 2 选 1-OT 协议

半诚实攻击模型下，基于公钥密码机制很容易构造 2 选 1-OT 协议。

图 3.2 给出了协议的构造方法。

参数：

- 两个参与方：发送方 S 和接收方 R 。
- S 的输入为秘密值 $x_0, x_1 \in \{0, 1\}^n$ ， R 的输入为选择比特 $b \in \{0, 1\}$ 。

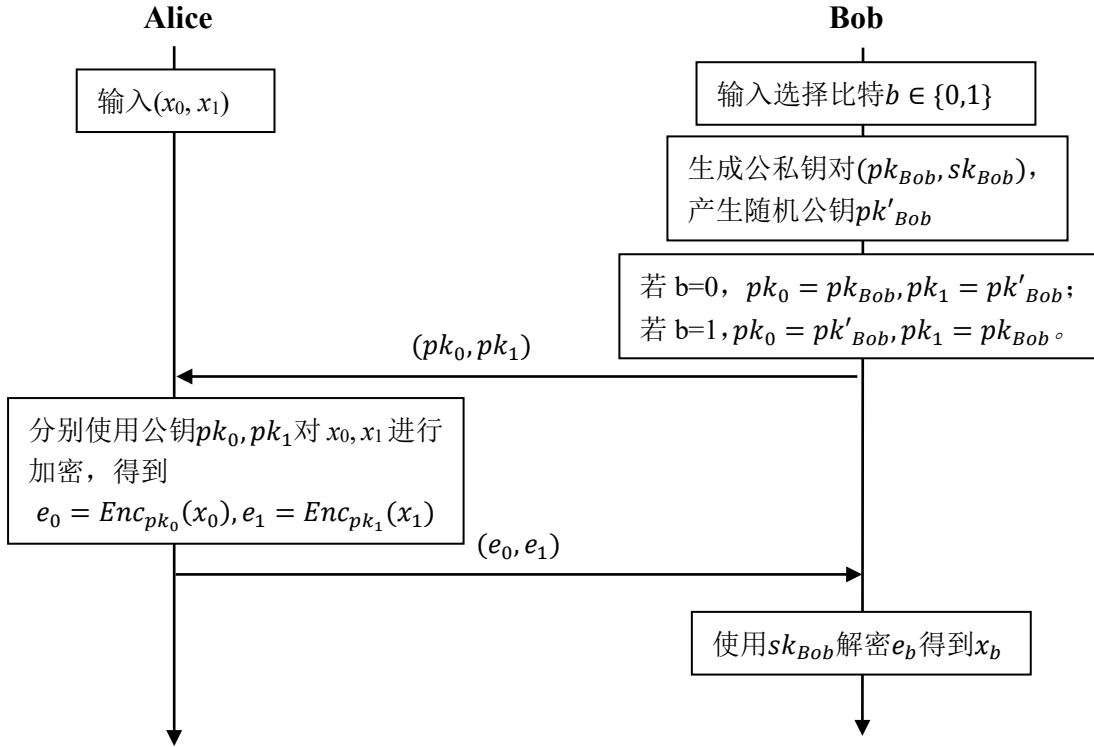
协议：

1. R 拥有一个公私钥对 (sk, pk) ，并从公钥空间中采样得到一个随机公钥 pk' 。如果 $b = 0$ ， R 将 (pk, pk') 发送给 S 。否则（如果 $b = 1$ ）， R 将 (pk', pk) 发送给 S 。
2. S 接收 (pk_0, pk_1) 并向 R 发送两个密文 $(e_0, e_1) = (Enc_{pk_0}(x_0), Enc_{pk_1}(x_1))$ 。

3. R 接收 (e_0, e_1) ，并用 sk 解密密文 e_b 。由于 R 不知道另一个密文所关联的密钥，因此 R 无法解密另一个密文。

图 3.2 基于公钥的半诚实安全 OT 协议

此协议假设存在一个公钥加密方案，可以在不获得对应私钥的条件下采样得到一个随机的公钥。此协议在半诚实攻击模型下是安全的。发送方 S 只能看到由 R 发送的两个公钥，因此 S 无法以超过 $1/2$ 的概率预测出 R 拥有哪个公钥所对应的私钥。因此，仿真者可以直接向 S 发送两个随机选择的公钥，从而仿真出 S 的视角。



安全性证明。回顾一下理想-现实范式，为了证明一个协议是安全的，在理想世界中的攻击者必须能够生成一个视角，此视角与真实世界中的攻击者视角不可区分，即：（1）攻击者能在理想世界中生成一个真实世界中的“仿真”攻击者视角，我们称此攻击者为仿真者，能说明存在这样一个仿真者，就能证明攻击者在现实世界中实现的所有攻击效果都可以在理想世界中实现；（2）如果现实世界中攻陷参与方所拥有的视角和理想世界中攻击者所拥有的视角不可区分，那么协议在半诚实攻击者的攻击下是安全的。具体的证明思路则是：如果任何现实模型攻击者 A 都存在一个理想模型攻击者 S （仿真器），对于任何输入，在现实模型下运行包含 A 的协议 π 的全局输出，它和在理想模型下运行包含 S 的 F 的全局输出是不可区分的，那么 π 至少和 F 一样安全。

接下来，我们以这个两方 OT 协议为例，再给出理想-现实范式的安全性证明思路。

定义 OT 协议的 **Ideal Functionality** F_{OT} 的描述如下：Alice 和 Bob 为了完成 2 选 1 茫然传输协议，Alice 向 F_{OT} 发送拥有的两个秘密 (x_0, x_1) 后、Bob 向 F_{OT} 发送选择比特 b 之后， F_{DH} 输出 x_b 给 Bob、输出 $\perp$ 给 Alice。

我们这里假定攻击者都是被动的，也就是“诚实且好奇”的攻击者，定义仿真器 S_{OT} 的仿真过程如下：

- ✧ 对于发送方的视角（**根据视角的定义，参与方的视角应包括其私有输入、随机带以及执行协议期间收到的所有消息所构成的消息列表**），因为仅提供两个秘密（私有输入）、接收两个公钥（协议期间收到的消息），这个很容易仿真。具体的，私有输入不需要仿真，两个公钥可以由仿真器本地随机生成。因为公钥都是随机生成的，仿真器里的公钥的分布和现实世界的分布是相同的，很明显现实世界和理想世界是无法区分的。补充说明：攻击者如果攻陷了发送方，他的目的是猜测两个公钥哪一个是有私钥的公钥，也就是接收方希望得到的消息 b 是哪一个。很明显，理想世界里仿真器随机生成的两个公钥是没有泄露任何信息的。
- ✧ 对于接收方 R 的视角，它的私有输入是 b 、接收两个密文（协议期间收到的消息）。接收方 R 可看到两个密文，其私钥只能用于解密其中一个密文。给定 R 的输入 b 和输出，同样很容易仿真 R 的视角。仿真器将生成公私钥对和一个随机公钥，并将仿真接收密文的仿真过程设置为：1）对于收到的秘密值 x_b 在所生成公私钥对下加密得到的密文 e_b ，2）明文 0 在随机公钥下加密得到的密文 e_{l-b} 。与协议的真实执行过程相比，只有 e_{l-b} 的生成过程有所区别，且根据加密方案的安全性要求，攻击者无法区分 0 和其他明文在同一公钥下加密得到的密文，因此仿真可以成功骗过攻击者，使攻击者无法判断是在真实世界中还是理想世界中，从而证明了协议的安全性。补充说明：攻击者如果攻陷了接收方，他的目的是破解另外一个秘密，而它的猜测能力被仿真器规约到区别是对 0 还是其它明文的加密上。

请注意，此半诚实安全协议无法抵御恶意接收方的攻击。接收方 R 可以简单地生成两个公私钥对 (sk_0, pk_0) 和 (sk_1, pk_1) ，并将 (pk_0, pk_1) 发送给 S 。这样一来， R 可以解密收到的两个密文，同时得到 x_1 和 x_2 。

说明：上述两方协议的理想-现实范式证明里，定义了一个仿真器、两个视角。事实上，安全多方计算协议的安全性证明通常使用一个仿真器，但是对于两方协议通常会定义两个仿真器，分别仿真发送方和接收方，具体证明过程与上面的视角的不可区分性相似。

（2）基于公钥的 k 选 1-OT 协议

常见的“2 选 1”不经意传输常被扩展为“ k 选 1”不经意传输。如图 2.3 所示，发送方 S 此时拥有 k 条隐私数据，接收方 R 拥有隐私的索引 $t \in \{0, 1, \dots, k-1\}$ 。在协议结束后，接收方 R 得到 x_t ，与“2 选 1”不经意传输一样，发送方 S 无法得知接收方 R 的隐私索引，接收方 R 也无法得到发送方 S 的其他数据。

根据上一小结给出的基本协议，容易得到“ k 选 1”不经意传输的实现方法，即：将协议中生成 1 个随机公钥改为生成 $k-1$ 个随机公钥。

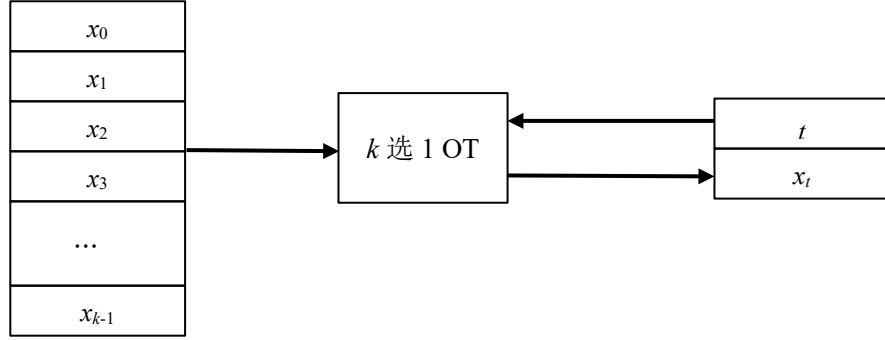


图 2.3 k 选 1 不经意传输

3.3.3 预计算 OT*

如上所述的基础 OT 协议中，发送方和接收方对每一个选择比特都要执行一系列公钥密码学操作。接收方会先发送一对公钥给发送方，发送方会将两个消息用两个公钥加密，发送给接收方，接收方执行公钥密码学的解密操作，获取选择的消息。

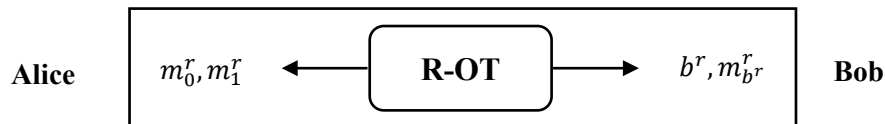
茫然传输的优化策略主要集中在两个点：第一，如何利用对称密码来减少公钥密码学操作数量；第二，如何利用预计算来提升在线时间的计算效率。

Beaver 在 1995 年提出了预计算 OT 的重要思想，首先给出了一个接收者随机的茫然传输 OT(receiver random oblivious transfer, RR-OT)协议，之后基于 RR-OT 结构，Beaver 设计了预计算 OT_2^1 协议。

预计算 OT 的在线阶段会消耗从预计算阶段的 RR-OT 实例获得的输出，整体效果即：将大量的计算放到预计算阶段，在确定了双方真实输入后，协议的线上执行阶段只需要执行少量的操作。

RR-OT: $(\perp, \perp) \mapsto ((r_0, r_1), (b, r_b))$ 。在 RR-OT 中，发送方和接收方都没有输入，协议执行结束后，发送方获得两个随机消息，而接收方获得一个随机比特 b 和对应的消息 r_b ，发送方完全不知道接收方获得哪个消息。实际 RR-OT 协议设计中，发送方的两个随机消息可以自行产生。

与 RR-OT 相比，我们之前给出的 OT 实例称为 2 选 1OT，记为 OT_2^1 。也就是说，发送方提供两个输入，接收方提供一个选择比特，发送方没有获得任何输出，但是接收方获取了选择比特对应的消息。即：OT: $((m_0, m_1), b) \mapsto (\perp, m_b)$ 。



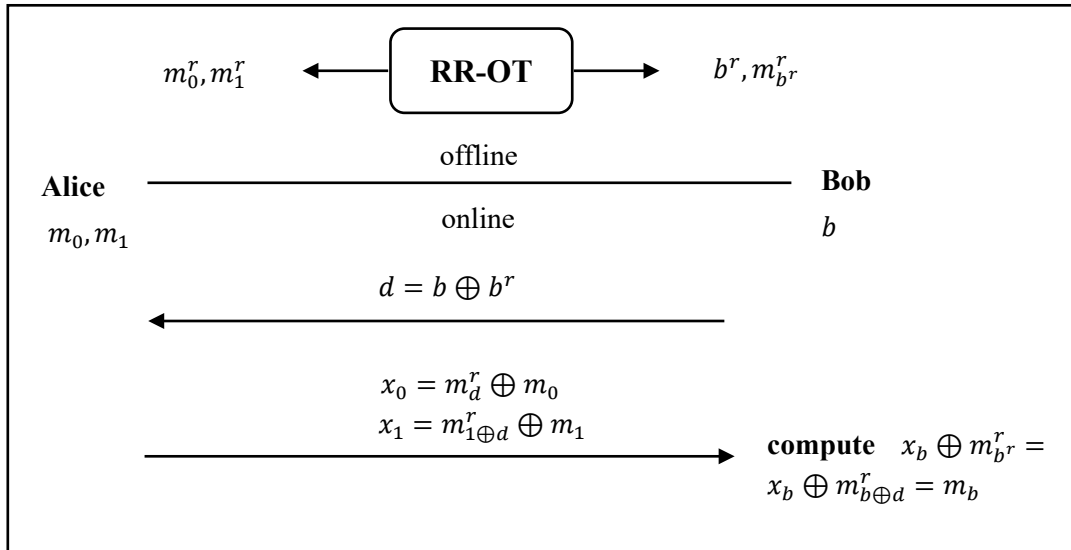
假定 RR-OT 为发送方 Alice 输出两个随机信息 m_0^r, m_1^r ，为接收方 Bob 输出随机的选择比特 b^r 和对应的随机消息 $m_{b^r}^r$ ，其中 $b^r \in \{0,1\}$ ，右上角的 r 表示随机选取。则可以设计预计算 OT_2^1 协议如下。

预计算 OT_2^1 协议。设 Alice 需要发送的信息为 m_0, m_1 ，Bob 的选择比特为 b 。假定已经在离线阶段运行 RR-OT，Alice 获得了两个随机信息 m_0^r, m_1^r ，接收方 Bob 获得了 $b^r, m_{b^r}^r$ 。

Beaver 去随机化的主要思想是 Alice 使用 RR-OT 的两个信息 m_0^r, m_1^r 一次性加密他要发送的信息 m_0, m_1 并将盲化结果 (x_0, x_1) 发送给 Bob。但是，为了确保 Bob 能正确解密，需要根据 Bob 拥有的选择比特选择合适的加密过程：

- ✧ 如果 $b = b^r$ ，Alice 发送的两个信息是 $x_0 = m_0^r \oplus m_0, x_1 = m_1^r \oplus m_1$ ，Bob 本地计算 $x_b \oplus m_{b^r}^r = x_b \oplus m_b^r = m_b$ ，即可得到选择比特 b 对应的比特秘密 m_b ，但是无法获得 m_{1-b} 。
- ✧ 如果 $b \neq b^r$ ，Alice 发送的两个信息是 $x_0 = m_1^r \oplus m_0, x_1 = m_0^r \oplus m_1$ ，Bob 本地计算 $x_b \oplus m_{b^r}^r = x_b \oplus m_{1 \oplus b^r}^r = m_b$ ，即可得到选择比特 b 对应的比特秘密 m_b 。

由于两种加密方式不同，Alice 并不知道 b 和 b^r 的值，无法判断该使用哪种加密方式。为了解决上述问题，容易想到的解决方案是让 Bob 告知 Alice 是否有 $b = b^r$ ，具体如下图所示：



上述过程就是一个在基本的 2 选 1OT 上使用随机数来实现的过程。其优点是在线阶段只需要异或运算，非常高效，但是缺点也很明显：（1）其每执行一次 OT 都需要进行离线阶段的随机数生成，这个开销非常昂贵；（2）一次离线阶段的 RR-OT 一次只能产生一个 OT 实例，这也给其应用带来困难。Beaver 等人在 1996 年，使用了混合加密的方式对 RR-OT 进行了改进，使得其离线阶段效率得到了提高，并且例证了 OT 扩展方案的可行性。

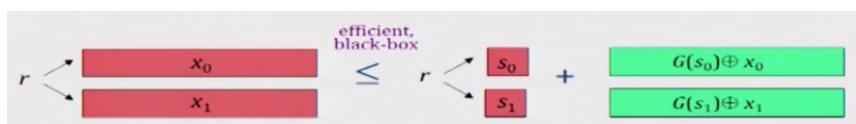
3.3.4 OT-扩展*

茫然传输扩展（OT Extension）协议：该协议的目的是通过执行固定次数的不经意协议，实现任意数量的不经意传输。

1. OT 长度扩展

这里思考一个问题，是否可以将 n 位长度的数据用更短的 OT 来完成传递？

假设我们已经有了用于传输短字符串信息的 OT，那么我们可以通过标准的伪随机数生成器生成用于传输长字符串信息的 OT，这一技术称为 OT 长度扩展。发送方将两个消息分别异或 $G(s_0)$ 或者 $G(s_1)$ ，将两个消息发送给接收方，接收方对两个种子 s_0 和 s_1 用一个基础 OT 获得即可。



2. Beaver 的非黑盒构造

1996 年，Beaver 提出了一种自举姚氏乱码电路协议，可以用少量公钥密码学操作生成多项式数量级的 OT 协议。其功能函数 F 的输入是来自接收方的少量比特串，但 F 将多项式数量级的 OT 协议的执行结果输出给接收方。

假设计算电路 C 的乱码电路协议要使用 m 个 OT 协议，其中 m 为 P_2 的输入比特数量。我们遵从 OT 协议的表示方法，称 P_1 （乱码协议中的电路生成方）为发送方 S ， P_2 （乱码电路中的电路求值方）为接收方 R 。令 m 表示现在需要批量执行的 OT 协议数量。 S 的输入为 m 对秘密值 $(x_1^0, x_1^1), \dots, (x_m^0, x_m^1)$ ， R 的输入为 m 比特长的选择比特串 $(b_1, \dots, b_m)$ 。

我们现在要构造出一个实现功能函数 F 的电路 C 。 R 提供给 F 的输入是随机选择的 λ 比特长字符串 r 。令 G 为一个伪随机生成器，可以将 λ 比特长随机数扩展到 m 比特长。 R 向 S 发送用随机字符串 $G(r)$ 加密的输入比特串 $b \oplus G(r)$ 。随后， S 向 F 提供的输入为 m 对秘密值 $(x_1^0, x_1^1), \dots, (x_m^0, x_m^1)$ 和一个 m 比特长字符串 $b \oplus G(r)$ 。给定 r ，函数 F 计算 m 比特长的扩展值 $G(r)$ ，解密 $b \oplus G(r)$ ，得到选择比特串 b 。 F 接下来只需要向 R 输出 b_i 所对应的秘密值 x_{b_i} 。在上述电路 C 中， R 作为电路求值方只需要向 F 提供 λ 比特长的输入，因此只需要用 λ 个（即常数个）OT 协议即可实现 m 个 OT 协议。

基于 Beaver 的非黑盒构造可以满足上述 RR-OT 协议，可以让 S 随机生成 m 对秘密值 $(x_1^0, x_1^1), \dots, (x_m^0, x_m^1)$ ， R 随机生成 m 比特长的选择比特串 $(b_1, \dots, b_m)$ 即可，然后通过上述构造电路 C 的方式，即可让 S 获得 m 对秘密值、 R 获得 m 对随机比特和对应的秘密。

3. OT 实例扩展

Beaver 给出的非黑盒构造方法非常简单，可以将执行 m 个 OT 协议所需的非对称密码学操作数量降低为 κ 次，其中 κ 为预先设定的安全参数。但是，Beaver 方案在实际中并不高效，因为方案要对一个非常大的乱码电路求值。回想一下，我们的目标是基于少量的 λ 个基础 OT 协议，只应用对称密码学操作实现 m 个有效的 OT 协议。下面我们描述 Ishai 等人在 2003 年提出的 OT 扩展 IKNP 协议。此协议可在半诚实攻击者的攻击下实现 m 个 2 选 1 OT 协议，用来安全地传输 m 个随机字符串。

IKNP 是第一个高效的 λ 个基础 OT 协议扩展为 $n(\gg \lambda)$ 个 OT 协议的工作。IKNP 协议和 Beaver 等人解决的问题实际上是相同的，都是为了让 Alice 获得消息对 (m_0, m_1) 和让 Bob 获得对应的 (r, m_r) 以便在线阶段 OT 的使用。具体如下：

假设 Alice 和 Bob 想生成 n 个 OT, Alice 是发送方，Bob 是接收方。

首先，生成两个秘密份额矩阵。Bob 随机构造长度为 n 的比特串 r ，这里 r 的每个比特都是每次 OT 的一个选择比特。然后将其看为一个 $n \times 1$ 的列向量，并对每个比特位进行按行扩展为 λ 位(重复编码扩展)，生成一个 $n \times \lambda$ 维的矩阵 R 。完成后进行秘密分享，生成两个秘密份额矩阵 (T, T') ， T 和 T' 相同的行异或为 1。若矩阵 R 的行向量全为 0，则 T, T' 在该行的元素相同，反之则不相同，如图：

r	
1	1 1 1 1 1 1 1 1
0	0 0 0 0 0 0 0 0
0	0 0 0 0 0 0 0 0
0	0 0 0 0 0 0 0 0
1	1 1 1 1 1 1 1 1
0	0 0 0 0 0 0 0 0
1	1 1 1 1 1 1 1 1
1	1 1 1 1 1 1 1 1
:	:

 $=$

1	1	0	1	1	0	0	1
1	0	1	1	0	1	0	0
0	1	1	0	0	0	1	1
1	1	0	1	1	0	1	1
1	1	0	1	0	1	1	0
1	0	1	0	1	0	0	0
0	0	0	0	0	1	0	1
0	0	1	1	1	0	1	0
:	:	:	:	:	:	:	:

 $\oplus$

0	0	1	0	0	1	1	0
1	0	1	1	0	1	0	0
0	1	1	0	0	0	1	1
1	1	0	1	1	0	1	1
0	0	1	0	1	0	0	1
1	0	1	0	1	0	0	0
1	1	1	1	1	0	1	0
1	1	0	0	0	1	0	1
:	:	:	:	:	:	:	:

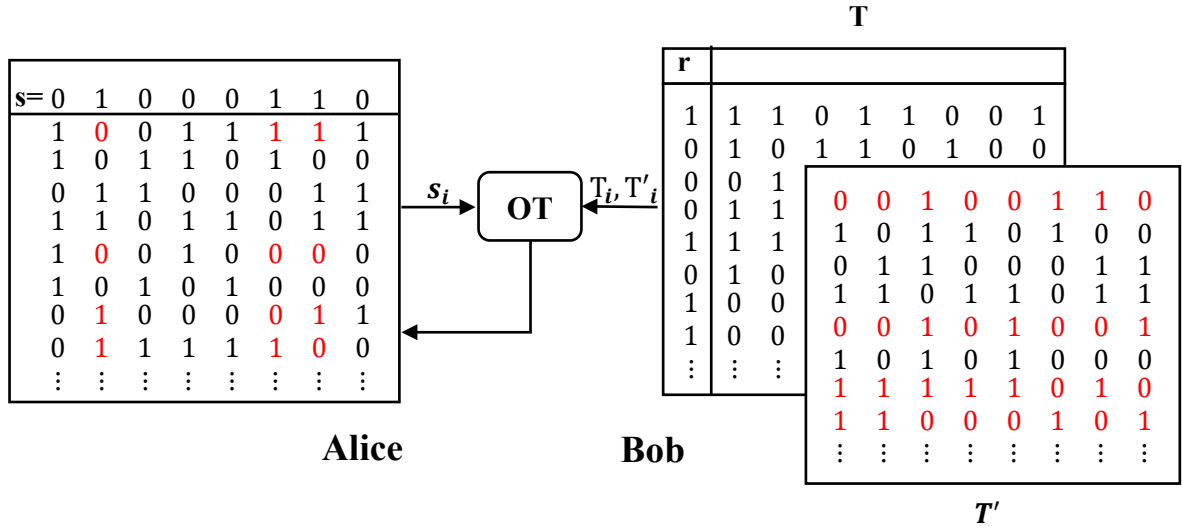
R
 T
 T'

接着，将行数据利用列传递降低交互次数。Alice 随机选取长度为 λ 的比特串 s ，然后双方交换身份，即 Alice 转变为接收方，Bob 转变为发送方，执行 λ 次基础 OT(次数取决于矩阵的列数)，将两个秘密份额矩阵 T, T' 的第 i 列作为输入的两个消息，Alice 作为接收方，以 s_i 为输入来选择两个秘密份额矩阵第 i 列的某一个来构成自己的矩阵 Q 的第 i 列，若 $s_i = 0$ ，则 Alice 获得矩阵 T 的第 i 列，反之则 Alice 获得矩阵 T' 的第 i 列。运行 λ 次后，Alice 获得矩阵 Q 。

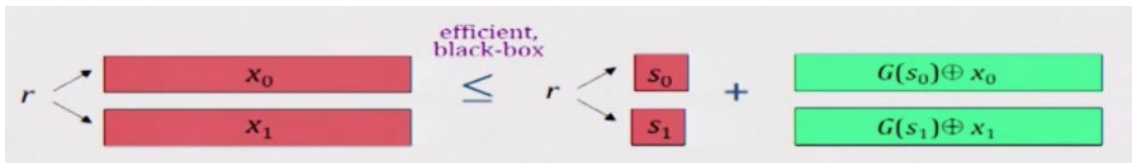
易知，当 $r_i = 0$ 时，两个秘密份额矩阵 T, T' 对应位置的行向量相等，所以 Alice 所得矩阵 Q 的行向量 q_i 与 Bob 的任意一个份额矩阵该位置对应的行向量 t_i 相同；当 $r_i = 1$ 时，Alice 所得矩阵 Q 的行向量 q_i 等于 Bob 的份额矩阵 T 该位置对应的行向量 t_i 与 s 的异或，即：

$$q_i = \begin{cases} t_i & \text{if } r_i = 0 \\ t_i \oplus s_i & \text{if } r_i = 1 \end{cases}$$

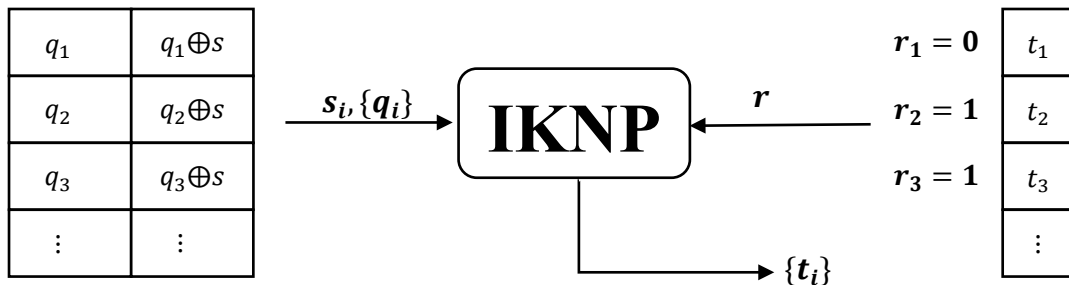
其中 t_i 是份额矩阵 T 的行向量，上述过程可以进一步抽象为： $q_i = t_i \oplus (r_i \wedge s_i)$ 。结果如下：



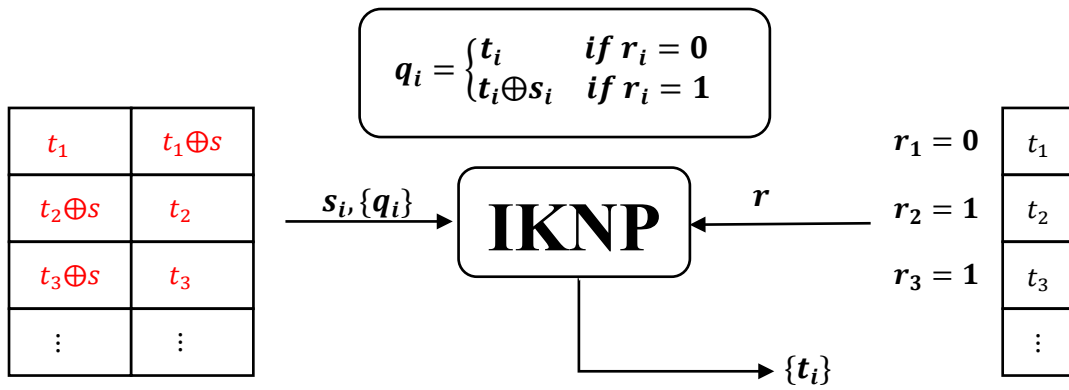
这里思考一个问题，是否可以将 n 位长度的数据用更短的 OT 来完成传递？很容易扩展 OT 协议的消息传输长度，只需要加密并发送两个 n 比特长的字符串，再用传输短字符串的 OT 协议发送正确的解密密钥即可。发送方将两个消息分别异或 $G(s_0)$ 或者 $G(s_1)$ ，将两个消息发送给接收方，接收方对两个种子 s_0 和 s_1 用一个基础 OT 获得即可。



第三，构造随机消息对和对应的选择比特与消息。上述过程后，Alice 获得了矩阵 Q ，加上自己拥有的选择串 s ，则 Alice 就有了消息对 $(q_i, q_i \oplus s)$ 。Bob 则拥有了相应的 r_i （选择比特）， t_i （某个消息），如下图所示：

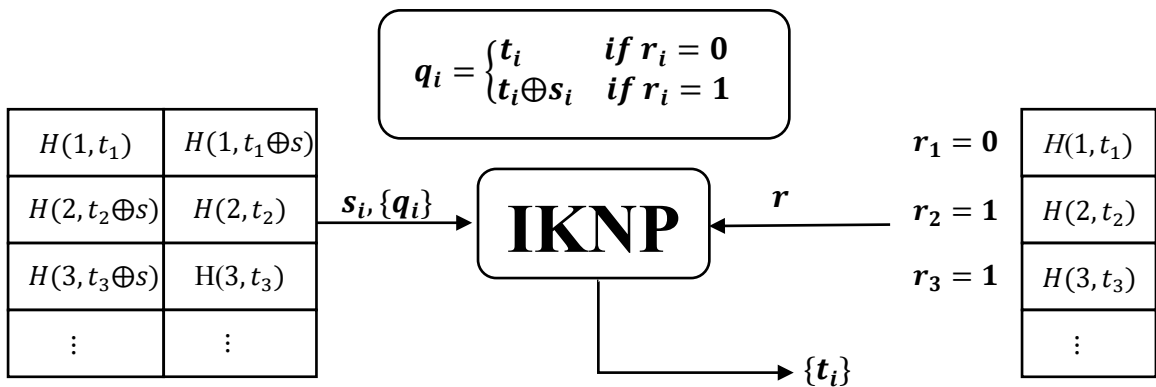


对于每个 i ，Bob 得到 t_i ，Alice 可以计算 q_i 和 $q_i \oplus s_i$ 。根据公式 $q_i = t_i \oplus (r_i \wedge s_i)$ ，我们可以将 q_i 和 $q_i \oplus s_i$ 根据 r_i 的不同，表示成下图中 t_i 与 s 的关系的形式。而从 Bob 的视角来看，他所拥有的 t_i 恰好是 Alice 秘密信息 q_i 和 $q_i \oplus s_i$ 中的一个。

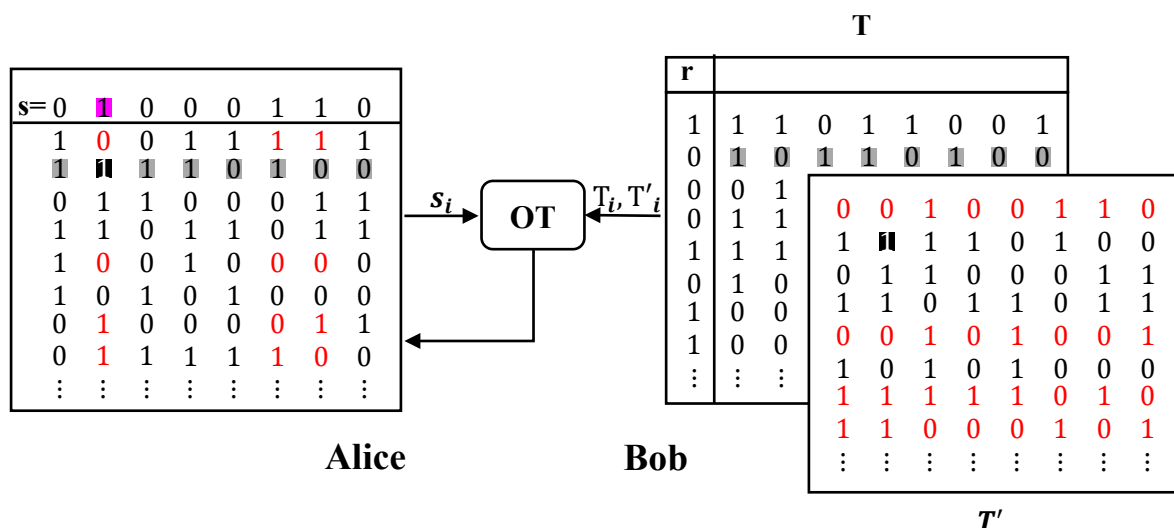


由此就已经可以明显的看出 Alice 拥有了随机的消息对，并且 Bob 拥有选择比特位和 Alice 消息对中的其中一个消息。这已解决开始想要解决的问题（两方获得用于在线阶段的消息队等）。效率方面，扩展矩阵是 $n \times \lambda$ 的矩阵，其中 $n \gg \lambda$ ，而基础-OT 阶段是按照列来进行，所以会进行 λ 次基础-OT，然而传输了 n 位的内容，并且不需要姚氏混淆电路。

完整 INKP 协议。但是仔细观察可以看到，上述的消息对之间重复的使用了同一个比特串 s ，这使得生成的消息之间存在相关性，所以必须解决这种相关性。本协议采用的方法是：使用一个随机预言机(Random Oracle)来解决相关性，用哈希函数 H 来实现随机预言机。对于消息 $t_1, t_2, \dots, t_n$ 和比特串 s ， H 使得 $H(t_1, s), H(t_2, s), \dots, H(t_n, s)$ 是独立伪随机的。从而解决上述问题。整个 INKP 协议如下图所示：



协议的安全性。安全性方面, INKP 协议是半诚实安全的。具体而言，Alice 可以是恶意的，但 Bob 只能是半诚实的，下面将介绍如果 Bob 是恶意的，INKP 协议会存在的问题。



若 Bob 在生成的某一个份额矩阵中修改 $r = 0$ 对应行向量的某一个比特位 (如 T' 中黑色方块由原来的 0 变为 1)，并且同时 Alice 对应的选择向量 s 对应的比特位 $s_i = 1$ (Alice 得到的矩阵中粉色方块位置)，那么就会造成 Alice 得到的向量与 Bob 具有的行向量不同 (灰色块所示的向量)。在后续协议执行过程中，例如哈希等，只要 Bob 检测到 Alice 的结果与自己的不同，就可以推断出该位 $s_i = 1$ ，相同则推断出 $s_i = 0$ 。如此一来，Bob 得到 Alice 选择串 s 的一个比特值。因此 IKNP 仅仅是半诚实安全。

3.4 混淆电路

3.4.1 姚氏百万富翁问题

姚期智 (Andrew Chi-Chih Yao) 是中国第一个也是目前为止唯一的图灵奖获得者。他为现代密码学打开了一道新的大门。其中一项重要贡献，就是安全多方计算。他提出了著名的姚氏百万富翁问题：假如有两个百万富翁 Alice 和 Bob 各有钱 i 和 j ，他们想知道谁的钱多，但是又不想把自己的钱数量告诉对方，怎样才能比大小呢？

下面介绍姚期智先生的解：

- (1) 先把问题简化，假如 i 和 j 是 1 到 10 之间的数。
- (2) 首先 Bob 挑选一个大整数 x ，然后用 Alice 的公钥 a 加密得到 $k = \text{Enc}(x)$ 。然后把数 k_{-j+1} 发给 Alice。
- (3) Alice 拿到这个数之后，计算以下这些数：

$$\text{Dec}(k_{-j+1}), \text{Dec}(k_{-j+2}), \text{Dec}(k_{-j+3}), \dots, \text{Dec}(k_{-j+10})$$

然后除以一个素数 p 取余得到：

$$z_1 = \text{Dec}(k-j+1)(\text{mod } p), z_2, z_3, \dots, z_{10}$$

因为 Alice 的钱是 i ，那么 Alice 做一下这个事情：

$$z_1, z_2, z_3, \dots, z_{i-1}, z_i = z_i + 1, z_{i+1} = z_{i+1} + 1, \dots$$

也就是将第 i 及后面的数都加 1，然后把这一串数字发给 Bob。

(4) Bob 只需要看第 j 个数字，如果等于 $x \pmod{p}$ 就说明 $i \geq j$ 否则说明 $i < j$ 。然后，Bob 把结果返给 Alice。

看着很复杂，实际上这个协议很简单。重点我们要去理解姚期智是怎么想的？

Bob 选择了一个非常大的数 x ，然后加密得到 k 。然后把 $k-j+1$ 发给了 Alice。这个时候 Bob 的信息已经藏在这个数据里面了，但是因为 x 很大，所以 Alice 没办法从 $k-j+1$ 之中推导出 j ，对 Alice 来说就是个随机数。

然后，Alice 计算了 10 个数， $\text{Dec}(k-j+\{1, 2, 3 \dots 10\})$ 。对 Alice 来说 $k-j+\{1, 2, 3 \dots 10\}$ 都是没有意义的随机数。但是，其中一个数是有意义的，那就是 $\text{Dec}(k-j+j)$ ，为什么呢？因为 $k-j+j=k$ ，而 k 是什么呢？ $k = \text{Enc}(x)$ 那么理所当然 $\text{Dec}(k)=x$ 。所以说这十个数对 Alice 没有意义，但是对 Bob 是有意义的，因为他知道第 j 个数就是 x 。

接下来，Alice 在这个十个数从第 i 个数开始都+1。再给 Bob，会发生什么？因为 Bob 知道第 j 个数是 x ，如果 Bob 收到的是 $x+1$ ，那么他知道，自己的数 j 肯定不小于 Alice 的 i ，反之则小于。

是不是 Alice 实际上巧妙地把自己的数 i 的信息放进去了这十个数当中，但是她知道这十个数除非是第 j 个数，否则其它的数对 Bob 没有任何意义。所以她不用担心自己的数 i 泄露（Dec 操作也是随机操作）。但是又可以比较大小。这就是这个协议的中心思想，是不是非常巧妙？

回过头来想想，你可以用几何的思维去想这个协议，Alice 和 Bob 都把信息藏到了两个不同的维度，Bob 藏在了单个数中 ($k-j+1$)，Alice 藏在了十个数中（第 i 个数之后加 1 了）。这就像两条直线，他们彼此对对方都没有任何意义。Bob 看不懂 Alice 的十个数，Alice 看不懂 Bob 的单个数。但是他们有个“交点”，在这个上 Alice 的数突然对 Bob 有了意义，从而 Bob 可以比较这两个数的大小了。这个“交点”就是“比较大小”。

那么我们是不是明白了 MPC 的中心思想，那就是彼此把数据 x, y 藏在自己的维度上，然后找到一个交点，就是我们需要共同计算的函数 $F(x, y)$ 。

3.4.2 姚氏混淆电路

Yao 的上述百万富翁求解的协议的交点仅限于比较大小，而其他的运算还没有支持。当然后来一个在此基础上一个更加伟大的密码学算法被发明出来了，那就姚氏乱码电路（Garbled Circuit, GC），是最著名、最广为人知的 MPC 协议。混淆电路的思想很简单，它源于一个事实：可以通过设计电路来对目标问题进行求解。电路可以通过与或非门实现任意一个函数，而多方计算的目标就是保护各方输入信息的情况下进行目标函数的计算。

一般认为姚氏 GC 协议具有最优的执行效率。我们要讲解的很多协议都是在姚氏 GC 协议的基础上构造得来的。虽然此协议的通信复杂度不是已有协议中最优的，但此协议的执行轮数是常数，避免协议设计本身引入较大的通信延迟。

参考《深入浅出》—增强混淆电路生成计算等方面的更加直观的描述和认识

1. 基本思想

回顾一下，为了要求解函数 $F(x, y)$ 的值，参与方 P_1 持有 $x \in X$ ，参与方 P_2 持有 $y \in Y$ 。这里的 X 和 Y 分别为 P_1 和 P_2 的输入域。

将函数表示为查找表。我们首先考虑一个输入域很小的函数 F 。由于 F 的输入域很小，我们可以很快地枚举出所有可能的输入对 (x, y) 。可以把函数 F 表示为一个包含 $|X| \cdot |Y|$ 行的**查找表 T** ，每行的条目为 $T_{x,y} = \langle F(x, y) \rangle$ ，即该表只有 1 列，共有 $|X| \cdot |Y|$ 行。只要能定位 $T_{x,y}$ ，就可以获得 $F(x, y)$ 的输出。

可以按照下述方式生成查找表： P_1 通过为每一个可能的输入 x 和 y 随机指定一个强密钥来加密 T 。也就是说，对于每一个 $x \in X$ 和每一个 $y \in Y$ ， P_1 将选择 $k_x \in_R \{0,1\}^\kappa$ 和 $k_y \in_R \{0,1\}^\kappa$ 。随后， P_1 同时使用两个密钥 k_x, k_y 加密 T 中相应的数据项 $T_{x,y}$ ，并将加密后（且经过随机置换）的查找表 $\langle \text{Enc}_{k_x, k_y}(T_{x,y}) \rangle$ 发送给 P_2 。

利用茫然传输隐藏行内访问的元素。我们现在的任务是让 P_2 （只能）解密与参与方输入相关联的数据项 $T_{x,y}$ 。具体实现方式是让 P_1 向 P_2 （茫然的）发送密钥 k_x 和 k_y ： P_1 已知自己的输入 x ，因此 P_1 只需要将密钥 k_x 直接发送给 P_2 即可（因为只是发送了一次性选的密钥，这个过程不会泄露 x ）； P_1 需要使用 $|Y|$ 选 1-OT 协议将 k_y 发送给 P_2 。一旦收到 k_x 和 k_y ， P_2 就可以使用这些密钥解密 $T_{x,y}$ ，得到输出 $F(x, y)$ 。

最重要的是， P_2 在此过程中无法获得任何其他信息。这是因为 P_2 只拥有一对密钥，只能打开（解密）查找表中的一个数据项。需要特别强调的是，单独使用 k_x 或 k_y 都不允许部分解密密文，甚至不能单独使用 k_x 或 k_y 判断某个密文是否是用 k_x 或 k_y 加密得到的。

安全求解函数 $F(x, y)$ 。由于置换表 T 是随机置换后发给 P_2 ，那么 P_2 如何知道 (x, y) 对应的数据项呢？因为该信息依赖于两个参与方的输入，所以该信息本身也是敏感的。

解决这个问题最简单的方法是在 T 的加密条目中编码一些附加信息。例如, P_1 可以在 T 的每一行字符串的末尾附加 σ 个 0。如果解密了错误的行, 则解密结果的末尾仅有很低的概率($p = \frac{1}{2^\sigma}$)包含 σ 个 0, 这样 P_2 就可以知道解密结果是错误的。虽然这个方法是可行的, 但是它对于 P_2 来说效率很低, 因为 P_2 平均要解密查找表 T 中至少一半的条目。

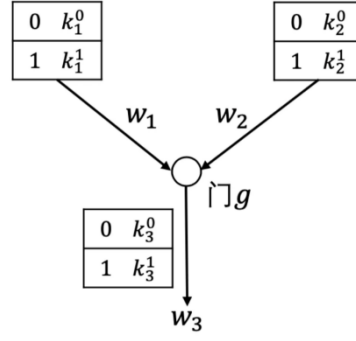
2. 构造方法

利用标识置换实现行的定位。1990 年, Beaver 等人提出了标识置换 (Point-and-Permute) 技术, 可以确定特定输入对应的密文数据项在置乱后的查找表中的位置。此方法的基本思想是将密钥的一部分 (即第一个密钥的后 $\lceil \log |X| \rceil$ 比特和第二个密钥的后 $\lceil \log |Y| \rceil$ 比特) 作为查找表 T 的置换标识, 标识密钥应该用于加密哪行 (或者哪个位置) 密文, 根据置换标识对加密后的查找表进行置换。在具体协议中, 可以设计为先确定查找表的置换标识, 然后根据这个置换标识完成置换。

为了避免查找表的各行在分配过程中产生冲突, P_1 必须保证置换标识不会在 k_x 的密钥空间和 k_y 的密钥空间中出现冲突, 可以通过多种方式实现这一点。严格来说, 密钥长度必须要达到相应的安全等级。因此, 参与方并不会直接把密钥中的一部分作为置换标识, 而是将置换标识附加在密钥之后, 使密钥满足所需的长度要求。在后续讨论中, 我们假定求值方已知要解密查找表中的哪一行。在描述协议时, 我们会根据上下文决定是否有必要明确指出把标识置换技术作为协议的一个组成部分。

管理查找表的大小。显然, 上述方案的效率较低, 因为查找表的大小与 F 的定义域大小呈线性关系。同时, 对于如单布尔电路门这样的小型函数, 其定义域的大小仅为 4, 用查找表表示此类小型函数是比较高效的。因此, 降低查找表的大小的思路是: 将 F 表示为布尔电路 C , 并用定义域大小为 4 的查找表求解每一个门的输出。对于布尔电路而言, 电路实现与或非即可实现完备, 可以模拟任意的函数。

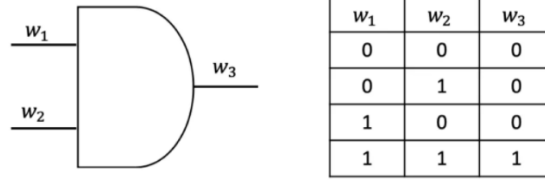
布尔电路的基础结构如下图所示, 门 g 可以是与门、或门等, 它接收两个输出, 输出一个结果。与前面的描述相同, P_1 生成密钥并加密查找表, P_2 在未知密钥与明文之间关系的条件下解密查找表。但在这种情况下, 我们不能向 P_2 披露中间门的明文输出。我们可以让中间门的输出也是与明文一一对应的密钥。由于求值方 P_2 不知道密钥与明文的关联关系, 因此这样可以达到隐藏中间门明文输出的目的。



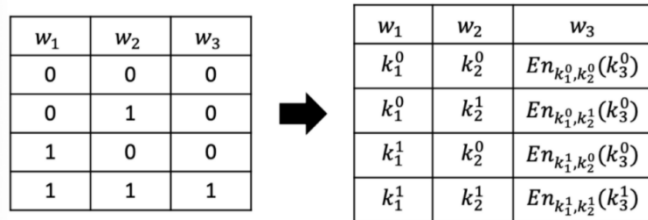
混淆电路基础结构

P_1 要为 C 的每一条导线 w_i 指定两个密钥 k_i^0 和 k_i^1 ，两个密钥分别与导线的两个可能的明文值相关联。我们称这些密钥为**导线标签**（Wire Label），称导线的明文值为**导线值**（Wire Value）。在对**电路求值**的过程中，根据计算过程中的输入，每条导线都将与一个特定的明文导线值和相应的导线标签相关联，我们称此明文导线值为**激活值**（Active Value），称与激活值对应的导线标签为**激活标签**（Active Label）。需要强调的是，求值方只知道激活标签，但不知道激活标签对应的激活值和非激活标签。可以这么理解，导线值和导线标签是相对于电路生成方的叫法，而激活值和激活标签是相对于电路求值方的叫法。

生成混淆电路。首先以与门为例，一个常见的与门及其真值表（Truth Table）如下图所示，将该与门的输入线记为 w_1, w_2 ，输出线记为 w_3 。

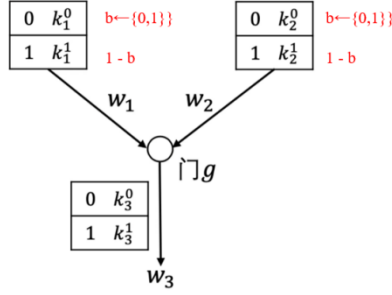


随机生成 6 个密钥 $k_1^0, k_1^1, k_2^0, k_2^1, k_3^0, k_3^1$ ，分别表示 w_1, w_2, w_3 这三条线为 0 和 1 时的两种情况。如 k_1^0, k_1^1 分别代表 w_1 为 0 和 w_1 为 1， k_3^0, k_3^1 分别代表 w_3 为 0 和 w_3 为 1。接着该门利用对称加密算法 $En()$ 生成 4 个密文 $c_{0,0}, c_{0,1}, c_{1,0}, c_{1,1}$ 。 $En_{a,b}(c)$ 表示用 a, b 作为加密密钥、使用加密算法 $En()$ 来加密 c ，即 $c_{a,b} = En_{a,b}(c) = En_a(En_b(c))$ 。



$T_G = \{c_{0,0}, c_{0,1}, c_{1,0}, c_{1,1}\}$ 就是 P_1 构建的初始查找表，查找表中的每一行条目都是门输出值所对应导线标签的密文。考虑安全性，需要对查找表的元素进行随机置乱。在查找表只包含 4 行密文的情况下，标识置换技术非常简单和高效。每个输入的置换标识只有 1 比特长，因此乱码表中的每行条目总共需要 2 比特长的置换标识。因此，在为每条导线选择对应的随

机密钥的时候，随机为其产生对应的置换比特（红色部分），进而任意两个密钥的加密就放置到对应的两个置换比特确定的位置即可。通常把置换后的查找表称为**乱码表**（Garbled Table）或乱码门（Garbled Gate），并将所有乱码表发送给 P_2 。



比如，如果为 k_1^0 随机选择的置换比特为 1，则 k_1^1 的置换比特则被设置为 0，同样，如果为 k_2^0 随机选择的置换比特为 0，则 k_2^1 的置换比特则被设置为 1，在这个情况下，产生的最后的乱码表就是 $T_G = \{c_{1,0}, c_{1,1}, c_{0,0}, c_{0,1}\}$ 。

对混淆电路求值。在收到输入密钥和乱码表后， P_2 开始对电路求值。对于 F 中属于 P_1 的输入导线， P_1 直接将对应的激活标签发送给 P_2 。对于 F 中属于 P_2 的输入导线， P_1 和 P_2 通过 2 选 1-OT 协议传输对应的激活标签。

假设门上两条线 w_1, w_2 的输入的值对为(0,1)，那么输入线对应的电路计算值为 (k_1^0, k_2^1) 。 k_1^0 是 P_1 直接发送给 P_2 的，而 k_2^1 是通过 2 选 1-OT 协议获得的。因为两个密钥各自蕴含了置换比特，所以，可以找到对应的要计算的密文 $c_{0,1}$ 。很重要的一点是，对一个乱码门的求解，允许求值方 P_2 得到输出 w_3 对应的激活标签，并在不知道中间激活标签所对应的激活值的条件下，利用中间激活标签继续做下一个电路门的输入，直到完成 F 的安全求值。

最终， P_2 完成乱码电路的求值，并得到与电路输出导线关联的密钥。 P_2 把得到的密钥发送给 P_1 解密，即可完成 F 的安全求值。

解码表。我们注意到， P_2 可以不用将导线标签发送给 P_1 解密，这样可以节省一轮通信过程。具体方法是让 P_1 在发送乱码电路的同时发送输出导线的解码表。解码表只是将输出导线的每个导线标签映射为对应的导线值（即相应的明文值）。此时，得到输出导线标签的 P_2 可以在解码表中直接查找导线标签所对应的导线值，得到明文输出。

3. 执行过程

图 1 形式化描述了姚氏乱码电路协议的生成过程，图 2 总结了姚氏乱码电路协议的执行过程。为了简化协议的描述，我们给出的是安全性依赖于随机预言机模型的变种协议，但应用较弱的伪随机函数（Pseudo-Random Function, PRF）存在性假设也足以完成姚氏乱码电路协议的构造。在加密乱码表中的各个条目时，协议使用了 H 表示的随机预言机。

参数：

- 实现函数 $\mathcal{F}$ 的布尔电路 $\mathcal{C}$ 。
- 安全参数 κ 。

生成乱码电路：

1. 生成导线标签。对于 $\mathcal{C}$ 的每一条导线 w_i ，随机选择导线标签：
 - (a) $w_i^b = (k_i^b \in_R \{0,1\}^\kappa, p_i^b \in_R \{0,1\})$ 。
 - (b) 使得 $p_i^b = 1 - p_i^{1-b}$ 。
2. 构造乱码电路。对于 $\mathcal{C}$ 的每一个门 G_i ，按照拓扑顺序执行下述步骤：

图 3.1 姚氏乱码电路协议：生成过程

- (a) 假设 G_i 是一个实现函数 $g_i: w_c = g_i(w_a, w_b)$ 的 2-输入布尔门，其中输入导线标签为 $w_a^0 = (k_a^0, p_a^0)$ ， $w_a^1 = (k_a^1, p_a^1)$ ， $w_b^0 = (k_b^0, p_b^0)$ ， $w_b^1 = (k_b^1, p_b^1)$ ，输出导线标签为 $w_c^0 = (k_c^0, p_c^0)$ ， $w_c^1 = (k_c^1, p_c^1)$ 。

- (b) 构造 G_i 的乱码表。 G_i 的输入值为 $v_a, v_b \in \{0,1\}$ ，输入值共有 2^2 种可能的组合。对于每一种组合，设

$$e_{v_a, v_b} = H(k_a^{v_a} \parallel k_b^{v_b} \parallel i) \oplus w_c^{g_i(v_a, v_b)}$$

根据输入导线标签上的置换标识对乱码表中的条目 e 排序，将条目 e_{v_a, v_b} 放置在位置 $\langle p_a^{v_a}, p_b^{v_b} \rangle$ 上。

3. 输出解码表。对于每一条输出导线 w_i (此导线也是门 G_j 的输出导线)，假设其对应的导线标签为 $w_i^0 = (k_i^0, p_i^0)$ ， $w_i^1 = (k_i^1, p_i^1)$ ，要为两个可能的导线值 $v \in \{0,1\}$ 创建解码表。设

$$e_v = H(k_i^v \parallel \text{"out"} \parallel j) \oplus v$$

(因为是逐比特执行异或运算，所以只需要使用 H 输出的最低位生成 e_v)。根据导线标签上的置换标识对解码表中的条目 e 排序，将条目 e_v 放置在位置 p_i^v 上 (因为 $p_i^1 = p_i^0 \oplus 1$ ，所以放置的位置不会发生冲突)。

图 3.1 (续)

参数：

- 参与方 P_1 和 P_2 ，其输入分别为 $x \in \{0,1\}^n$ 和 $y \in \{0,1\}^n$ 。
- 实现函数 $\mathcal{F}$ 的布尔电路 C 。

协议：

1. P_1 的角色为乱码电路生成方，执行图 3.1 所示的步骤。随后， P_1 将得到的乱码电路 $\hat{C}$ (包括输出解码表) 发送给 P_2 。
2. 对于 P_1 需要提供输入值的导线， P_1 向 P_2 发送输入值所对应的激活标签。
3. 对于 P_2 需要提供输入值的导线 w_i ， P_1 作为发送方， P_2 作为接收方，两方执行 OT 协议：
 - (a) P_1 的秘密值为导线上的两个导线标签， P_2 的选择比特输入值为 P_2 在此条导线上的输入值。
 - (b) OT 协议执行完毕后， P_2 得到导线的激活标签。
4. P_2 从输入导线的激活标签开始，按照拓扑顺序逐门对收到的 $\hat{C}$ 求值。
 - (a) 对于乱码表为 $T = (e_{0,0}, \dots, e_{1,1})$ ，输入激活标签为 $w_a = (k_a, p_a)$ 和 $w_b = (k_b, p_b)$ 的门 G_i ， P_2 计算输出激活标签 $w_c = (k_c, p_c)$ ：
$$w_c = H(k_a \parallel k_b \parallel i) \oplus e_{p_a, p_b}$$
5. 应用输出解码表得到输出。当对 $\hat{C}$ 的所有门完成求值后， P_2 把第二个密钥设置为“out”，解码最终的输出门，得到明文计算结果。 P_2 将得到的计算结果发送给 P_1 ，双方均将计算结果作为协议的输出。

图 3.2 姚氏乱码电路协议：求值过程

3.4.3 示例

对参与运算的双方，从参与者的视角可以分为电路产生者(circuit generator)与电路执行者(circuit evaluator)。混淆电路有两个阶段：

- ✧ 电路产生阶段：电路产生者将待计算的函数转化为电路。参与运算的双方先就需要安全计算的目的依靠专有编程语言(DSL)或相关编程语言扩展等进行编程，然后针对实现计算的程序进行编译，生成电路文件；
- ✧ 电路执行阶段：电路执行者安全地计算电路。利用不经意传输 (Oblivious Transfer, OT)、加密等密码学原语等执行电路，在不泄露电路的输入和中间结果的前提下，完成电路的计算。

1. 电路生成

混淆电路要求要计算的函数能被逻辑电路表示，所以如何将函数转化为一个逻辑电路是关键的一步。混淆电路的构造从门开始先加密一个门再延伸到加密整个电路。

我们以著名的姚氏百万富翁问题为例，尝试将这一大小比较的函数转化为电路。

逻辑电路。我们不妨认为 Alex 和 Bob 的财富是用二进制表示的一个整数

$a_n a_{n-1} \dots a_1, b_n b_{n-1} \dots b_1$ ，其中 $a_i, b_i \in \{0,1\}$ 。我们可以用归纳法来判断它们的大小。

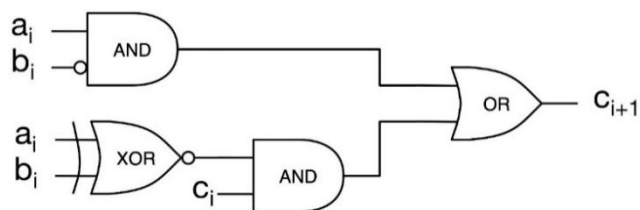
我们定义变量 c_i :

$$c_i := \begin{cases} 1 & \text{if } a_{i-1} \dots a_1 > b_{i-1} \dots b_1 \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

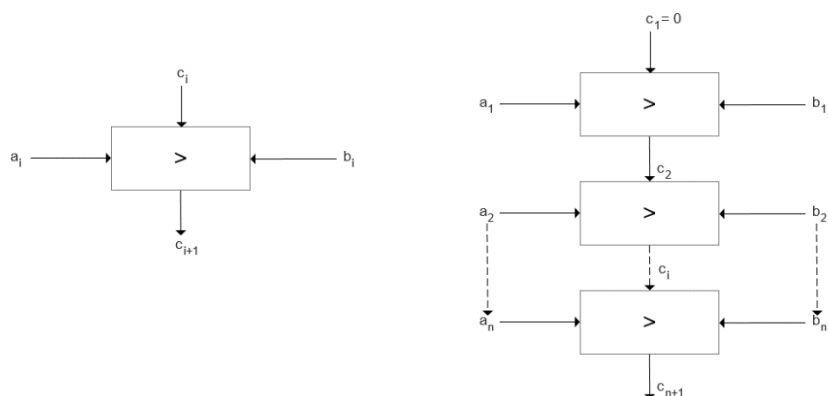
以及其初始值 $c_1 = 0$ 。在已知 a_i, b_i, c_i 的情况下, c_{i+1} 可以做如下推导。

$$c_{i+1} = 1 \Leftrightarrow (a_i > b_i) \text{ or } (a_i = b_i \text{ and } c_i = 1) \quad (2)$$

公式(2)描述的逻辑也很直接, 即 $a_i \dots a_1 > b_i \dots b_1$ 的充分必要条件是 $a_i > b_i$, 或 $a_i = b_i$ 且 $a_{i-1} \dots a_1 > b_{i-1} \dots b_1$ 。通过这个方法, 我们可以依次获得 $c_2, c_3, \dots, c_{n+1}$ 。对应到逻辑电路, 由于 $a_i, b_i, c_i \in \{0,1\}$, $a_i > b_i$ 可以表示为 $a_i \text{ and } \sim b_i$, $a_i = b_i$ 可以表示为 $\sim(a_i \text{ xor } \sim b_i)$, 其中 $\sim$ 表示取反, 公式(2)可以转化成如下逻辑电路(图中圆圈表示取反)。



我们将上述电路封装成一个三个输入(a_i, b_i, c_i)、一个输出(c_{i+1})的模块。我们将 n 个这样的模块串联起来, 就完成了判断 $a_n a_{n-1} \dots a_1 > b_n b_{n-1} \dots b_1$ 的电路。

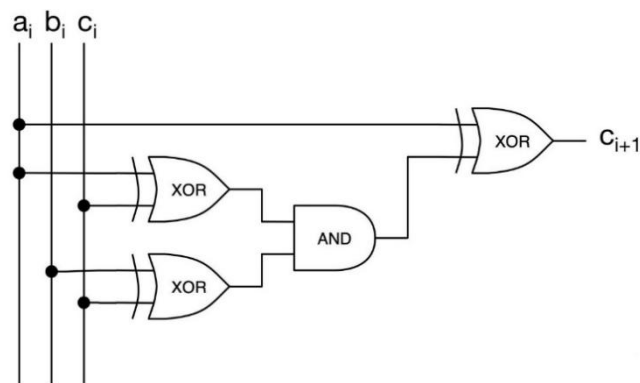


该电路中, c_{n+1} 为整个电路的输出。当输出是1时, $a_n a_{n-1} \dots a_1 > b_n b_{n-1} \dots b_1$ 成立。

上文提到的电路用到了多处取反, 并不是最优的。通过观察, 我们不难发现 $c_{i+1} = a_i \oplus [(a_i \oplus c_i) \wedge (b_i \oplus c_i)]$ 。

a_i	b_i	c_i	c_{i+1}	$a_i \oplus [(a_i \oplus c_i) \wedge (b_i \oplus c_i)]$
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	0	0	0
1	1	1	1	1

电路可以由此优化为:



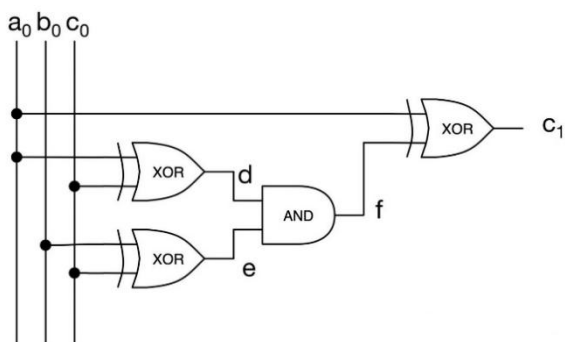
2. 电路求解

首先说明一下，在这个例子中，我们将隐藏掉置换标识的处理，假定采用了最初附加特定位的低效方法，也不严格按照 3.2.3 所述的 GC 协议执行过程，我们更关注于解释整个求解过程。

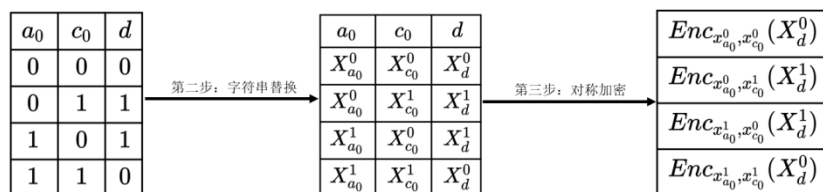
Step 1: Alice 生成混淆电路

首先，Alice 基于上述电路生成对应的混淆电路。生成过程主要分四步。

第一步，Alice 对电路中的每一线路（Wire）进行标注。如下图所示，Alice 一共标注了七条线路，包括模块的输入输出 $W_{a_0}, W_{b_0}, W_{c_0}, W_{c_1}$ ，和模块内的中间结果 W_d, W_e, W_f 。对于每一条线路 W_i ，Alice 生成两个长度为 k 的字符串 X_i^0, X_i^1 。这两个字符串分别对应逻辑上的 0 和 1。这些生成的标注会在 Step 2 有选择性地发给 Bob，但 Bob 并不知道 X_i^0, X_i^1 对应的逻辑值。



第二步，Alice 对电路中的每一个逻辑门的真值表用 X_i^0, X_i^1 进行替换，由 X_i^0 替换 0，由 X_i^1 替换 1。比如电路图中左上方的 XOR 门的输入是 a_0, c_0 ，输出是 d ，对应的真值表可以做如下转换。



第三步，Alice 对每一个替换后的真值表的输出进行两次对称加密，加密的密钥是真值表对应行的两个输入。比如真值表的第一行是 $X_{a_0}^0, X_{c_0}^0, X_d^0$ ，我们就用 $X_{a_0}^0, X_{c_0}^0$ 来加密 X_d^0 ，生成 $Enc_{x_{a_0}^0, x_{c_0}^0}(X_d^0)$ 。

Step 2: Alice 和 Bob 通信

第一步，Alice 将她的输入对应的字符串发送给 Bob。比如 $a_0 = 1$ ，那 Alice 会发送 $X_{a_0}^1$ 给 Bob。由于 Bob 不知道 $X_{a_0}^1$ 对应的逻辑值，也就无从知晓 Alice 的秘密了。

第二步，Bob 通过不经意传输（OT）协议从 Alice 获得他的输入对应的字符串。不经意传输保证了 Bob 在 $\{X_{b_0}^0, X_{b_0}^1\}$ 中获得一个，且 Alice 不知道 Bob 获得了哪一个。所以 Alice 也就无从知晓 Bob 的 b_0 了。

最后，Alice 将所有逻辑门的乱码表都发给 Bob。在这个例子中，一共有四个乱码表。

Step 3: Bob evaluate 生成的混淆电路

Alice 和 Bob 通信完成之后，Bob 便开始沿着电路进行解密。

因为 Bob 拥有所有输入的标签和所有乱码表，他可以逐一对每个逻辑门的输出进行解密。在这个例子中，假设 Bob 拥有的输入标签为 $X_{a_0}^1, X_{b_0}^1, X_{c_0}^0$ 。他可以：

1. 对于电路图左上方的 XOR，用 $X_{a_0}^1, X_{c_0}^0$ 解密获得 X_d^1 ；
2. 对于电路图左下方的 XOR，用 $X_{b_0}^1, X_{c_0}^0$ 解密获得 X_e^1 ；
3. 对于电路图中间的 AND，用 X_d^1, X_e^1 解密获得 X_f^1 ；
4. 对于电路图右侧的 XOR，用 $X_{a_0}^1, X_f^1$ 解密 $X_{c_1}^0$ 。

值得注意的是，由于乱码表每一行的密钥都不同，所以 Bob 只能解密其中一行。而且 Bob 并不知道解密出来的 $X_d^1, X_e^1, X_f^1, X_{c_1}^0$ 对应的逻辑值，也就无从获得更多信息了。而 Alice 全程不参与 Bob 的解密过程，所以也无法获得更多信息。

Step 4: 共享结果

最后 Alice 和 Bob 共享结果，Alice 分享 $\{X_{c_1}^0, X_{c_1}^1\}$ 或者 Bob 分享 $X_{c_1}^1$ ，双方就能获得电路输出的逻辑值了。

3.5 秘密共享

3.5.1 基本概念

1. 定义

秘密共享 (Secret Sharing) 是一种将秘密分割存储的密码技术, 目的是阻止秘密过于集中, 以达到分散风险和容忍入侵的目的, 是信息安全和数据保密中的重要手段。目前, 秘密共享已成为一种重要密码学工具, 在诸多多方安全计算协议中被使用, 例如拜占庭协议、多方隐私集合求交协议、阈值密码学等。

秘密分享方案的安全性有很多不同的定义方法。下面的定义是在 Beimel 和 Chor (Beimel and Chor, 1992) 定义的基础上修改而来的。

定义 令 D 为秘密值所在域, 令 D_1 为秘密份额所在域。令 $Shr: D \rightarrow D_1^n$ 为秘密分配算法 (可能是随机性算法), $Rec: D_1^k \rightarrow D$ 为秘密重构算法, 其中 D_1^n 表示 n 个秘密份额、 D_1^k 表示 k 个秘密份额。 (t, n) -秘密分享方案包含一对算法 (Shr, Rec) , 满足下述两个性质:

- **正确性。** 令 $(s_1, s_2, \dots, s_n) = Shr(s)$, 则:

$$Pr[\forall k \geq t, Rec(s_{i_1}, \dots, s_{i_k}) = s] = 1$$

- **完美隐私性。** 任意包含少于 t 个秘密份额的集合都不会在信息论层面上泄漏与秘密值相关的任何信息。

我们在多数情况下使用的都是 (n, n) -秘密分享方案, 即拥有全部 n 个秘密份额是重建出秘密值的充分必要条件。

2. 分类

基于秘密分发, 我们将秘密共享分为基于位运算的加性秘密共享和基于线性代数的线性秘密共享。

依据秘密重构的条件或者秘密分享的份额数量等, 又可以将秘密共享分为如下类别:

- 门限秘密共享: 任意大于等于阈值的参与方集合可重构出秘密。
- 多重秘密共享: 参与方的子秘密可以多次使用, 分别恢复多个共享秘密
- 多秘密共享: 一次共享, 共享多个秘密, 且子秘密可以重复使用
- 可验证秘密共享: 可通过公共变量验证自己子秘密的正确性
- 动态秘密共享: 允许添加或删除参与方, 定期或不定期更新参与方的子秘密, 还允许在不同的时间恢复不同的秘密。

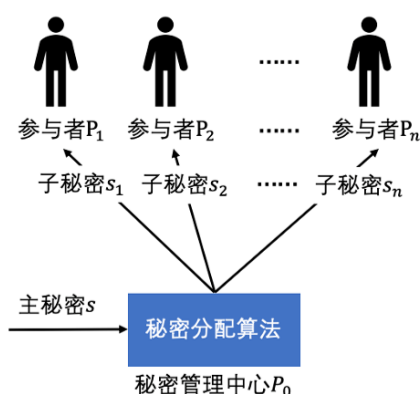
3.5.2 Shamir 方案

1. 门限秘密共享

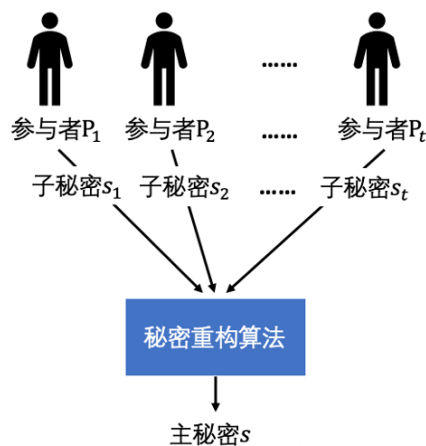
在 1979 年，Shamir 提出了门限秘密共享算法。

设 t 和 n 为两个正整数，且 $t \leq n$ 。 n 个需要共享秘密的参与者集合为 $P = \{P_1, \dots, P_n\}$ ，一个 (t, n) 门限秘密共享体制是指：假设 $P_1, \dots, P_n$ 要共享同一个秘密 s ，将 s 称为主秘密，有一个秘密管理中心 P_0 来负责对 s 进行管理和分配。秘密管理中心 P_0 掌握有秘密分配算法和秘密重构算法，这两个算法均满足重构要求和安全性要求。

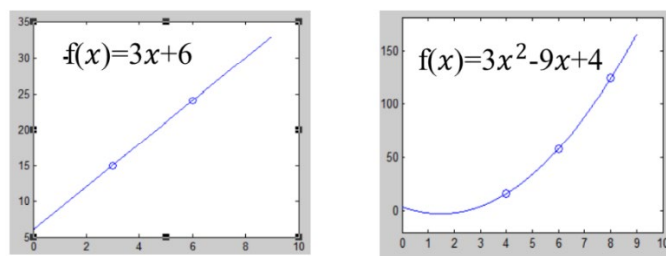
秘密分配。 秘密管理中心 P_0 首先通过将主秘密 s 输入秘密分配算法，生成 n 个值，分别为 $s_1, \dots, s_n$ ，称 $s_1, \dots, s_n$ 为子秘密。然后秘密管理中心 P_0 分别将秘密分配算法产生的子秘密 $s_1, \dots, s_n$ 通过 P_0 与 P_i 之间的安全通信信道秘密地传送给参与者 P_i ，参与者 P_i 不得向任何人泄露自己所收到的子秘密 s_i 。



秘密重构。 门限值 t 指的是任意大于或等于 t 个参与者 P_i ，将各自掌握的子秘密 s_i 进行共享，任意的一个参与者 P_i 在获得其余 $t - 1$ 个参与者所掌握的子秘密后，都可独立地通过秘密重构算法恢复出主秘密 s 。而即使有任意的 $n - t$ 个参与者丢失了各自所掌握的子秘密，剩下的 t 个参与者依旧可以通过将各自掌握的子秘密与其他参与者共享，再使用秘密重构算法来重构出主秘密 s 。安全性要求是指任意攻击者通过收买等手段获取了少于 t 个的子秘密，或者任意少于 t 个参与者串通都无法恢复出主秘密 s ，也无法得到主秘密 s 的信息。



2. Shamir 方案



Shamir 于 1979 年，基于多项式插值算法设计了 Shamir(t,n)门限秘密共享体制。构造思路：

- 平面上不同的两个点唯一的确定平面上的一条直线，即一次多项式
- 平面上不同的三个点唯一的确定平面上的一条二次多项式
- 一般的，设 $\{(x_1, y_1), \dots, (x_k, y_k)\}$ 是平面上 k 个不同的点构成的点集，那么在平面上存在唯一的 $k-1$ 次多项式 $f(x) = a_0 + a_1x + \dots + a_{k-1}x^{k-1}$ 通过这 k 个点。
- 若把秘密 s 取做 $f(0)$ ， n 个份额取做 $f(i) (i = 1, \dots, n)$ ，那么利用其中任意 k 个份额就可以重构 $f(x)$ ，从而得到秘密 $s = f(0)$ 。

(1) 秘密分配算法

Shamir 于 1979 年，基于多项式插值算法设计了 Shamir(t,n)门限秘密共享体制，它的秘密分配算法如下：

首先假设 $\mathbb{F}_q$ 为 q 元有限域， q 是素数且 $q > n$ 。 $P = \{P_1, \dots, P_n\}$ 是参与者集合， P 共享主秘密 s ， $s \in \mathbb{F}_q$ ，秘密管理中心 P_0 按如下所述的步骤对主秘密 s 进行分配，为了可读性起见，以下公式均略去了模 q 操作：

1. P_0 秘密的在有限域 $\mathbb{F}_q$ 中随机选取 $t-1$ 个元素，记为 $a_1, \dots, a_{t-1}$ ，并取以 x 为变元的多项式 $f(x) = a_{t-1}x^{t-1} + \dots + a_1x^1 + s = s + \sum_{i=1}^{t-1} a_i x^i$ 。
2. 对于 $1 \leq i \leq n$ ， P_0 秘密计算 $y_i = f(i)$ 。
3. 对于 $1 \leq i \leq n$ ， P_0 通过安全信道秘密地将 (i, y_i) 分配给 P_i 。

(2) 秘密重构算法

依据 (t, n) 门限共享体制，秘密重构需要 t 个参与方都将所拥有的秘密分享出来，以便求解。通常有两类方法：解方程法和多项式插值法。

算法一：解方程法

解方程法比较通俗，即 t 个方程可以确定 t 个未知数，而这 t 个未知数即为包括主秘密 s 在内的多项式 $f(x)$ 的各项系数。

如果参与者 $P_1, \dots, P_t$ 掌握了子秘密 $f(1), \dots, f(t)$ ，解方程：

$$\begin{aligned} a_{t-1}1^{t-1} + \dots + a_11^1 + s &= f(1) \\ a_{t-1}2^{t-1} + \dots + a_12^1 + s &= f(2) \\ &\vdots \\ a_{t-1}n^{t-1} + \dots + a_1n^1 + s &= f(t) \end{aligned}$$

即可求解出系数 $a_{t-1}, \dots, a_1, s$ 。

算法二：多项式插值法

另一种方式是使用多项式插值法进行重构主秘密。

假设这 t 个子秘密分别为 (x_i, y_i) ，其中 $y_i = f(x_i)$ ， $i = 1, \dots, t$ 且 $i \neq j$ 时 $x_i \neq x_j$ 。参与者 $P_1, \dots, P_t$ 共同计算：

$$\begin{aligned} h(x) = & y_1 \frac{(x-x_2)(x-x_3)\dots(x-x_t)}{(x_1-x_2)(x_1-x_3)\dots(x_1-x_t)} + y_2 \frac{(x-x_1)(x-x_3)\dots(x-x_t)}{(x_2-x_1)(x_2-x_3)\dots(x_2-x_t)} + \dots \\ & + y_t \frac{(x-x_1)(x-x_2)\dots(x-x_{t-1})}{(x_t-x_1)(x_t-x_2)\dots(x_t-x_{t-1})} \end{aligned}$$

显然， $h(x)$ 是一个 $t-1$ 次的多项式，且因为 $i \neq j$ 时 $x_i \neq x_j$ ，每个加式的分母均不为零，因此对于 $i = 1, \dots, t$ ， $y_i = h(x_i) = f(x_i)$ 。

又根据多项式的性质，如果存在两个最高次均为 $t-1$ 次的多项式，这两个多项式在 t 个互不相同的点所取的值得均相同，那么这两个多项式相同，即 $h(x) = f(x)$ 。

因此，参与者 P_i 通过分享各自秘密，共同计算 $h(0) = f(0) = s$ ，即可恢复主秘密 s 。

(3) 举例说明：(3,5)门限方案

设 $t = 3$ ， $n = 5$ ， $q = 19$ ， $s = 11$ 。随机选取系数 $a_1 = 2$ ， $a_2 = 7$ ，则：

$$f(x) = 7x^2 + 2x + 11 \text{ mod } 19$$

计算可知， $f(1) = 1$ ， $f(2) = 5$ ， $f(3) = 4$ ， $f(4) = 17$ ， $f(5) = 6$

若已知 $f(2)$, $f(3)$, $f(5)$:

- 使用解方程法解如下方程组:

$$(a_2 \times 2^2 + a_1 \times 2 + s) \bmod 19 = f(2) = 5$$

$$(a_2 \times 3^2 + a_1 \times 3 + s) \bmod 19 = f(3) = 4$$

$$(a_2 \times 5^2 + a_1 \times 5 + s) \bmod 19 = f(5) = 6$$

可解得 $a_1 = 2$, $a_2 = 7$, $s = 11$ 。

- 使用多项式插值法, 由拉格朗日插值公式可知:

$$f(x) = 5 \frac{(x-3)(x-5)}{(2-3)(2-5)} + 4 \frac{(x-2)(x-5)}{(3-2)(3-5)} + 6 \frac{(x-2)(x-3)}{(5-2)(5-3)} = 7x^2 + 2x + 11$$

故 $s = f(0) = 11$ 。

3. 插值原理

已知 $f(x)$ 在区间 $[a, b]$ 上一组 $n + 1$ 个不同点 $a \leq x_0, x_1, \dots, x_n \leq b$ 上的函数值 $y_i = f(x_i), (i = 0, 1, 2, \dots, n)$, 构造满足插值条件 $L_n(x_i) = y_i (i = 0, 1, 2, \dots, n)$ 的次数不超过 n 次的多项式 $L_n(x) = \sum_{i=0}^n \varphi_i(x) f(x_i) \approx f(x)$, 其中 $\varphi_i(x) (i = 0, 1, 2, \dots, n)$ 是次数不超过 n 次的多项式。

两点拉格朗日插值。 设已知两个不同节点 x_0, x_1 上的函数值 $f(x_0) = y_0, f(x_1) = y_1$, 构造满足插值条件 $L_1(x_i) = y_i (i = 0, 1)$ 的次数不超过一次的多项式 $L_1(x) = \varphi_0(x)y_0 + \varphi_1(x)y_1$, 其中 $\varphi_i(x) (i = 0, 1)$ 是次数不超过 1 次的多项式。

实际上就是构造一次的多项式 $\varphi_0(x)$ 和 $\varphi_1(x)$ 并满足:

$$L_1(x_0) = \varphi_0(x_0)y_0 + \varphi_1(x_0)y_1 = y_0$$

$$L_1(x_1) = \varphi_0(x_1)y_0 + \varphi_1(x_1)y_1 = y_1$$

令 $\varphi_0(x)$ 和 $\varphi_1(x)$ 满足如下条件显然可以使上述两式成立:

	x_0	x_1
$\varphi_0(x)$	1	0
$\varphi_1(x)$	0	1

由于 $\varphi_0(x)$ 和 $\varphi_1(x)$ 均为一次多项式, 因此可求得:

$$\varphi_0(x) = \frac{x - x_1}{x_0 - x_1}, \varphi_1(x) = \frac{x - x_0}{x_1 - x_0}$$

因此有

$$L_1(x) = \varphi_0(x)y_0 + \varphi_1(x)y_1 = \frac{x - x_1}{x_0 - x_1} y_0 + \frac{x - x_0}{x_1 - x_0} y_1$$

这就是两点拉格朗日插值公式, 用它来作为函数 $y = f(x)$ 的近似。显然 $L_1(x)$ 是次数不超过一次的多项式, 且满足:

$$L_1(x_0) = \varphi_0(x_0)y_0 + \varphi_1(x_0)y_1 = 1 \cdot y_0 + 0 \cdot y_1 = y_0$$

$$L_1(x_1) = \varphi_0(x_1)y_0 + \varphi_1(x_1)y_1 = 0 \cdot y_0 + 1 \cdot y_1 = y_1$$

多点拉格朗日插值。 我们可以用同样的方法建立多点的拉格朗日插值公式。设已知 $n + 1$ 个不同节点 $x_0, x_1, \dots, x_n$ 上的函数值 $f(x_0) = y_0, f(x_1) = y_1, f(x_2) = y_2, \dots, f(x_n) = y_n$ ，构造满足插值条件 $L_n(x_i) = y_i (i = 0, 1, 2, \dots, n)$ 的次数不超过 n 次多项式

$$L_n(x) = \varphi_0(x)y_0 + \varphi_1(x)y_1 + \dots + \varphi_n(x)y_n$$

其中 $\varphi_i(x) (i = 0, 1, \dots, n)$ 是次数为 n 的多项式。

同前述方法，令 $\varphi_0(x), \varphi_1(x), \dots, \varphi_n(x)$ 满足如下条件：

	x_0	x_1	$\dots$	x_n
$\varphi_0(x)$	1	0	$\dots$	0
$\varphi_1(x)$	0	1	$\dots$	0
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$\varphi_n(x)$	0	0	$\dots$	1

可求得：

$$\varphi_i(x) = \frac{(x - x_0)(x - x_1) \dots (x - x_{i-1})(x - x_{i+1}) \dots (x - x_n)}{(x_i - x_0)(x_i - x_1) \dots (x_i - x_{i-1})(x_i - x_{i+1}) \dots (x_i - x_n)} = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

$$L_n(x) = \sum_{i=0}^n \varphi_i(x)y_i = \sum_{i=0}^n \left(\prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j} \right) y_i$$

这就是 $n + 1$ 点拉格朗日插值公式。

3.5.3 基于秘密共享的多方计算

1. 同态特性

同态秘密共享的思想：在保护各秘密数据的约束之下，以份额做加法或乘法运算的结果为输入，能由恢复算法恢复出多个秘密的和或积。（同态加密是指参与者在本地拥有的两个秘密份额上进行加法和乘法计算，进而可以通过秘密重构的方法，完成运算结果的恢复，就是明文消息加或者乘之后的结果。）

我们将以 Shamir 门限秘密共享方案为例，重点讨论秘密共享方案的加法同态、乘法同态（其他还包括除法同态、幂乘同态和比较同态）。这些同态的性质使得秘密共享能够被广泛地应用在不同的场景中。

（1）加法同态

假设输入的秘密 a 和秘密 b 已经通过随机多项式 $f_a(x)$ 和 $f_b(x)$ 利用 Shamir 门限体制共享给了各个参与者， $f_a(x) = m_{t-1}x^{t-1} + \dots + m_1x^1 + a$ ， $f_b(x) = n_{t-1}x^{t-1} + \dots + n_1x^1 + b$ ，

其中 $m_{t-1}, \dots, m_1, n_{t-1}, \dots, n_1$ 是随机的多项式系数。参与者 P_i 掌握输入 a 的子秘密 a_i 和输入 b 的子秘密 b_i 。

令 $f_c(x) = f_a(x) + f_b(x) = (m_{t-1} + n_{t-1})x^{t-1} + \dots + (m_1 + n_1)x^1 + (a + b)$ ，每个参与者 P_i 独立计算 $c_i = a_i + b_i, 1 \leq i \leq n$ ，即可得到经过多项式 $f_c(x)$ 的 n 个点 $(1, c_1) \dots (n, c_n)$ 。可以将 $f_c(x)$ 看作是为秘密 $a + b$ 进行 Shamir 秘密共享构建的多项式， $c_1, \dots, c_n$ 看作是秘密 $a + b$ 的 n 份秘密份额，那么就可以通过多项式插值法或者解方程重构出多项式 $f_c(x)$ ，进而计算 $f_c(0) = a + b$ ，得到秘密 a 和秘密 b 的和 $a+b$ 。

子秘密可以直接相加，是因为对于 $c_i = a_i + b_i = f_a(i) + f_b(i) = (m_{t-1} + n_{t-1})i^{t-1} + \dots + (m_1 + n_1)i + a + b$ ，多项式的次数并没有发生变化，新的多项式 $f_a(x) + f_b(x)$ 的最高次数依旧为 $t - 1$ 。

因此， t 个参与者共享他们所掌握的秘密份额以及计算出来的 c_i ，即可根据 t 个方程解 t 个未知数，解出 $a + b$ 。当然，参与者也可以使用拉格朗日插值法求解出 $a + b$ 。这样就实现了加法同态性，即对秘密份额做加法运算可以重构出秘密的和。

(2) 乘法同态

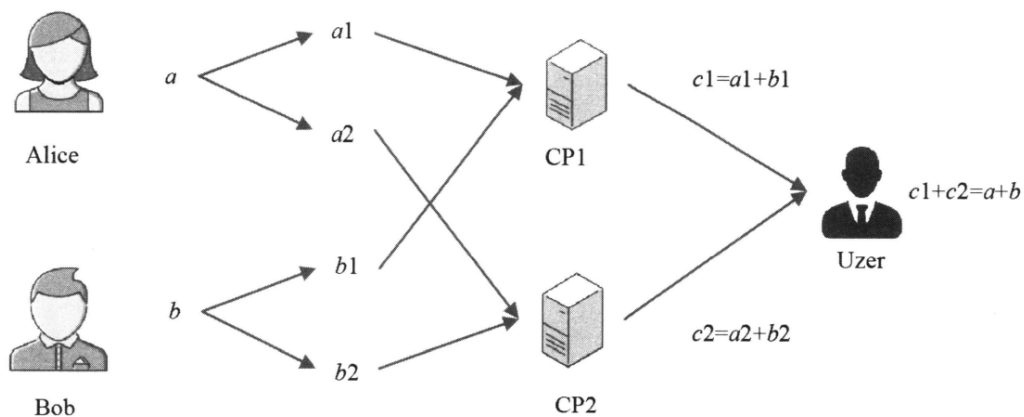
两个秘密 a 和 b 通过 (t, n) 门限秘密共享方案在不同参与方中共享，参与方可以利用 $(2t - 1, n)$ 门限秘密共享方案计算秘密乘积的共享份额。然而，Shamir 秘密共享具有受限的乘法同态性质，其受限来自于 2 个 $t - 1$ 次多项式相乘其结果是 $2(t - 1)$ 次多项式，根据拉格朗日插值定理需要 $2(t - 1) + 1$ 个秘密份额才能恢复出多项式。如果 $2(t - 1) + 1 > n$ ，那将会没有足够的秘密份额进行插值。因为多项式相乘，导致最高次项次数会发生变化，因此，我们可以通过增加其他计算的机制，使得 Shamir 秘密共享满足乘法同态的性质。

离散对数拥有如下性质： $\log_g ab = \log_g a + \log_g b$ ，根据该性质，我们可以对秘密 a 和秘密 b 的对数 $\log_g a$ 和 $\log_g b$ 进行 Shamir 秘密分享， $f_a(x) = m_{t-1}x^{t-1} + \dots + m_1x^1 + \log_g a$ ， $f_b(x) = n_{t-1}x^{t-1} + \dots + n_1x^1 + \log_g b$ ，其中 $m_{t-1}, \dots, m_1, n_{t-1}, \dots, n_1$ 是随机的多项式系数。那么通过加法同态，可以得到秘密 a 、秘密 b 的对数 $\log_g a$ 、 $\log_g b$ 的和 $\log_g a + \log_g b = \log_g ab$ ，即秘密 a 和秘密 b 的乘积的对数，进一步做指数运算就可以得到秘密的积 ab 。

其他如除法同态、幂乘同态和比较同态，也可以通过对秘密共享方案的设计而实现，本书不再赘述。

2. 多方计算示例

那么，如何利用秘密共享进行隐私计算呢？这里先以 $(2, 2)$ -门限的加法为例，假设 Alice 和 Bob 为输入方，Alice 拥有隐私输入 a ，Bob 拥有隐私输入 b ，Uzer 为结果使用方，设计两个计算方 CP1 和 CP2，如图所示。计算过程如下（实际过程比较复杂，以下描述主要用于辅助理解其基本思想）。

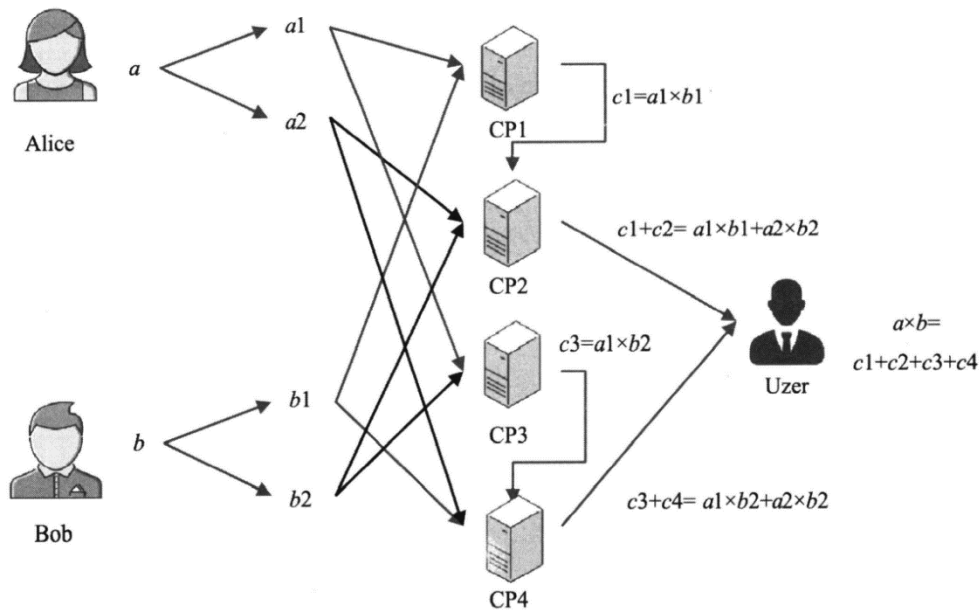


秘密共享的加法示例

- 1) Alice 方将隐私输入 a 随机拆分成 $a1$ 和 $a2$ ，并且使得 $a=a1+a2$ ，然后将 $a1$ 发送给 CP1，将 $a2$ 发送给 CP2。
- 2) Bob 方将隐私输入 b 随机拆分成 $b1$ 和 $b2$ ，并且使得 $b=b1+b2$ ，然后将 $b1$ 发送给 CP1，将 $b2$ 发送给 CP2。
- 3) CP1 计算 $a1+b1=c1$ ，并将 $c1$ 发送给 Uzer。
- 4) CP2 计算 $a2+b2=c2$ ，并将 $c2$ 发送给 Uzer。
- 5) Uzer 计算 $c1+c2$ 就可以获得 $a+b$ 两数之和（易知 $c1+c2=a1+b1+a2+b2=a+b$ ），并且计算过程中计算方无法获知隐私输入以及隐私输出的具体值，数据输出方也无法获知隐私输出的具体值。（这句话感觉有问题，应该是结果使用方无法获知隐私输入的具体值或是数据输入方无法获知隐私输出的具体值？）

再以乘法为例，假设 Alice 和 Bob 为输入方，Alice 拥有隐私输入 a ，Bob 拥有隐私输入 b ，Uzer 为结果使用方，设计 4 个计算方 CP1、CP2、CP3 和 CP4，如图所示，计算过程如下。

- 1) Alice 方将隐私输入 a 随机拆分成 $a1$ 和 $a2$ ，并且使得 $a=a1+a2$ ，然后将 $a1$ 发送给 CP1 和 CP3，将 $a2$ 发送给 CP2 和 CP4。
- 2) Bob 方将隐私输入 b 随机拆分成 $b1$ 和 $b2$ ，并且使得 $b=b1+b2$ ，然后将 $b1$ 发送给 CP1 和 CP4，将 $b2$ 发送给 CP2 和 CP3。
- 3) CP1 计算 $a1 \times b1=c1$ ，并将 $c1$ 发送给 CP2。
- 4) CP2 计算 $a2 \times b2=c2$ ，并将 $c1+c2$ 发送给 Uzer。
- 5) CP3 计算 $a1 \times b2=c3$ ，并将 $c3$ 发送给 CP4。
- 6) CP4 计算 $a2 \times b1=c4$ ，并将 $c3+c4$ 发送给 Uzer。
- 7) Uzer 计算 $c1+c2+c3+c4$ 就可以获得 a 和 b 之积（易知 $c1+c2+c3+c4=a1 \times b1+a2 \times b2+a1 \times b2+a2 \times b1=(a1+a2) \times (b1+b2)=a \times b$ ），并且计算过程中计算方无法获知隐私输入以及隐私输出的具体值，数据输出方也无法获知隐私输出的具体值。



秘密共享的乘法示例

在理论层面，很多密码学家用非常有限的运算操作定义 MPC 协议，涉及的运算操作包含模整数加法和乘法、逐比特的与运算和异或运算。这两类运算操作都是图灵完备的，任何函数都可以用这两种运算操作表示。举例来讲，学习过计算机组成原理的读者可知定点数除法可以通过加减交替法来实现。在实际中，只有加法和乘法，或只有逐比特的与运算和异或运算是不够的，我们还需要除法、比较等运算操作以及在此之上建立的基础函数库，以提高函数表达的效率。一些实用的隐私计算框架（比如 JIFF 框架）都支持实现除法等常用的运算操作。

3.5.4 应用实践

1. python 安装

一般情况下，Ubuntu 系统存在预装的 Python3。

(1) 查看已安装的 python 版本

```
Python3 --version
```

```
cpz2000@cpz2000-virtual-machine:~/桌面$ python3 --version
Python 3.8.10
```

(2) 如果没有 python，使用 APT 软件包管理工具安装

使用以下命令安装 python，需要保证处于 root 账户下或使用 sudo 命令提权

```
[sudo] apt install python3
```

2. Shamir 秘密共享

(1) 在 Ubuntu 的 home 文件夹下建立文件夹 secret_share，进入该文件夹后，编写文件 shamir.py 如下：

```
import random
#快速幂计算  $a^b \pmod p$ 
def quickpower(a,b,p):
    a=a%p
    ans=1
    while b!=0:
        if b&1:
            ans=(ans*a)%p
        b>>=1
        a=(a*a)%p
    return ans

#构建多项式: x0 为常数项系数, T 为最高次项次数, p 为模数, fname 为多项式名
def get_polynomial(x0,T,p,fname):
    f=[]
    f.append(x0)
    for i in range(0,T):
        f.append(random.randrange(0,p))
    #输出多项式
    f_print=fname+'='+str(f[0])
    for i in range(1,T+1):
        f_print+='+'+str(f[i])+'x^'+str(i)
    print(f_print)
    return f

#计算多项式值
def count_polynomial(f,x,p):
    ans=f[0]
    for i in range(1,len(f)):
        ans=(ans+f[i]*quickpower(x,i,p))%p
    return ans

#重构函数 f 并返回 f(0)
def restructure_polynomial(x,fx,t,p):
    ans=0
    #利用多项式插值法计算出 x=0 时多项式的值
    for i in range(0,t):
        fx[i]=fx[i]%p
        fxi=1
        #在模 p 下,  $(a/b) \pmod p = (a*c) \pmod p$ , 其中 c 为 b 在模 p 下的逆元,  $c=b^{(p-2)} \pmod p$ 
        for j in range(0,t):
            if j !=i:
                fxi=(-1*fxi*x[j]*quickpower(x[i]-x[j],p-2,p))%p
        fxi=(fxi*fx[i])%p
        ans=(ans+fxi)%p
    return ans
```

```

if __name__ == '__main__':
    #设置模数 p
    p=1000000007
    print(f'模数 p: {p}')

    #输入门限(t,n)以及秘密值 s1,s2
    n=int(input("请输入参与者的个数 n:"))
    t=int(input("请输入需要几个人参与者才可以恢复秘密:"))
    s1=int(input("请输入秘密 s1:"))
    s2=int(input("请输入秘密 s2:"))

    #秘密共享阶段:
    #为两个秘密 s1,s2 分别构造随机多项式 f1(x)和 f2(x)
    print('为秘密 s1,s2 构建随机多项式')
    f1=get_polynomial(s1,t-1,p,'f1')
    f2=get_polynomial(s2,t-1,p,'f2')

    #选择 n 个互不相同的随机值
    x=[]
    for i in range(0,n):
        temp=random.randrange(0,p)
        while temp in x:
            temp=random.randrange(0,p)
        x.append(temp)

    #计算得到 shares=[xi,f1(xi),f2(xi)], 其中 f1(xi)为秘密 s1 的秘密份额, f2(xi)为秘密 s2 的秘密份额
    shares=[]
    for i in range(0,n):
        shares.append([])
        shares[i].append(x[i])
        shares[i].append(count_polynomial(f1,x[i],p))
        shares[i].append(count_polynomial(f2,x[i],p))
    #输出秘密份额
    print('s1 秘密份额:')
    for i in range(0,n):
        print(f'({shares[i][0]},{shares[i][1]})')
    print('s2 秘密份额:')
    for i in range(0,n):
        print(f'({shares[i][0]},{shares[i][2]})')

    #将秘密分享给 n 个参与者, 第 i 个参与者得到 shares[i]

    #秘密重构阶段:
    #任意选取 t 个人
    Party=[]
    for i in range(0,t):
        temp=random.randint(0,n-1)
        while temp in Party:

```



```

        temp=random.randint(0,n-1)
        Party.append(temp)
    print('参与重构秘密的参与者:',Party)

    #t 个参与方分别将自己手中 s1 和 s2 的秘密份额相加，得到 s1+s2 的秘密份额
    shares_s1s2_x=[]
    shares_s1s2_y=[]
    print('s1+s2 的秘密份额为: ')
    for i in range(0,t):
        shares_s1s2_x.append(x[Party[i]])
        shares_s1s2_y.append((shares[Party[i]][1]+shares[Party[i]][2])%p)
        print(f'({shares_s1s2_x[i]}, {shares_s1s2_y[i]})')
    s1s2=restructure_polynomial(shares_s1s2_x,shares_s1s2_y,t,p)
    print(f'重构得到的秘密值为: {s1s2}')

```

(2) 打开终端，输入命令：

```
python3 shamir.py
```

(3) 运行结果如下：

```

cpz2000@cpz2000-virtual-machine:~$ python3 shamir.py
模数p: 1000000007
请输入参与者的个数n:5
请输入需要几个人参与才可以恢复秘密:3
请输入秘密s1:12
请输入秘密s2:4
为秘密s1,s2构建随机多项式
f1=12+985830208x^1+673404879x^2
f2=4+11937260x^1+730530753x^2
s1秘密份额:
(372329276,437513075)
(721382926,146990342)
(897670962,642418673)
(172706133,280863217)
(930379766,193403852)
s2秘密份额:
(372329276,366221276)
(721382926,285614877)
(897670962,499166437)
(172706133,573612279)
(930379766,722035225)
参与重构秘密的参与者: [4, 3, 2]
s1+s2的秘密份额为:
(930379766,915439077)
(172706133,854475496)
(897670962,141585103)
重构得到的秘密值为: 16

```

3. 投票统计

假设有三个同学需要对班里的优秀干部 Alice、Bob、Charles、Douglas 进行投票，最后统计各个班干部获得的票数。这个时候就可以利用秘密共享将各个投票方的投票分享出去并进行隐私求和计算。

以计算 Alice 的票数为例，三个参与方的隐私输入为 0 或 1，0 表示不投票给 Alice，1 表示投票给 Alice。

1) 在 ubuntu 下打开终端，执行如下命令，建立文件 vote.py

touch vote.py

2) 将如下代码复制粘贴到 vote.py

```
import random
#快速幂计算  $a^b \pmod p$ 
def quickpower(a,b,p):
    a=a%p
    ans=1
    while b!=0:
        if b&1:
            ans=(ans*a)%p
        b>>=1
        a=(a*a)%p
    return ans

#构建多项式: x0 为常数项系数, T 为最高次项次数, p 为模数, fname 为多项式名
def get_polynomial(x0,T,p,fname):
    f=[]
    f.append(x0)
    for i in range(0,T):
        f.append(random.randrange(0,p))
    #输出多项式
    f_print='f'+fname+'='+str(f[0])
    for i in range(1,T+1):
        f_print+=' '+str(f[i])+'x^'+str(i)
    print(f_print)
    return f

#计算多项式值
def count_polynomial(f,x,p):
    ans=f[0]
    for i in range(1,len(f)):
        ans=(ans+f[i]*quickpower(x,i,p))%p
    return ans

#重构函数 f 并返回 f(0)
def restructure_polynomial(x,fx,t,p):
    ans=0
    #利用多项式插值法计算出 x=0 时多项式的值
    for i in range(0,t):
        fx[i]=fx[i]%p
        fxi=1
        #在模 p 下,  $(a/b) \pmod p = (a*c) \pmod p$ , 其中 c 为 b 在模 p 下的逆元,  $c=b^{(p-2)} \pmod p$ 
        for j in range(0,t):
            if j !=i:
                fxi=(-1*fxi*fx[j]*quickpower(x[i]-x[j],p-2,p))%p
        fxi=(fxi*fx[i])%p
        ans=(ans+fxi)%p
    return ans

if __name__ == '__main__':
    #设置模数 p
    p=1000000007
    print(f'模数 p: {p}')
```

```

#输入门限(t,n)
    n=int(input("请输入参与者的个数 n:"))
    t=int(input("请输入需要几个人参与者才可以恢复秘密:"))
#三个投票方将自己的秘密值 s[0],s[1],s[2]分别秘密共享给 n 个参与方
s=[]#三个投票方的秘密值
f=[]#三个秘密值对应的多项式
shares_x=[]#三个秘密值对应的秘密份额
#选择 n 个互不相同的随机值
for i in range(0,n):
    temp=random.randrange(0,p)
    while temp in shares_x:
        temp=random.randrange(0,p)
    shares_x.append(temp)
shares_y=[]
for i in range(0,3):
    s.append(int(input(f'第{i+1}个投票方输入自己的投票值: ')))
    #输出多项式及秘密份额
    print(f'第{i+1}个投票方的投票值的多项式及秘密份额: ')
    f.append(get_polynomial(s[i],t-1,p,str(i+1)))
    temp=[]
    for j in range(0,n):
        temp.append(count_polynomial(f[i],shares_x[j],p))
        print(f'({shares_x[j]},{temp[j]}')
    shares_y.append(temp)
#在 n 个参与者中任选 t 个人重构求和之后的值
#任意选取 t 个人
Party=[]
for i in range(0,t):
    temp=random.randint(0,n-1)
    while temp in Party:
        temp=random.randint(0,n-1)
    Party.append(temp)
print('参与重构秘密的参与者:',Party)
#t 个参与方分别将自己手中 s[0]、s[1]和 s[2]的秘密份额相加,得到 s[0]+s[1]+s[2]
的秘密份额
shares_s123_x=[]
shares_s123_y=[]
for i in range(0,t):
    shares_s123_x.append(shares_x[Party[i]])
    temp=0
    for j in range(0,3):
        temp+=shares_y[j][Party[i]]
    shares_s123_y.append(temp)
s123=restructure_polynomial(shares_s123_x,shares_s123_y,t,p)
print(f'得票结果为: {s123}')

```

3)执行如下命令

```
python3 vote.py
```

结果如下

```
cpz2000@cpz2000-virtual-machine:~/桌面$ touch vote.py
cpz2000@cpz2000-virtual-machine:~/桌面$ python3 vote.py
模数p: 1000000007
请输入参与者的个数n:4
请输入需要几个人参与者才可以恢复秘密:3
第1个投票方输入自己的投票值: 0
第1个投票方的投票值的多项式及秘密份额:
f1=0+995837472x^1+342946838x^2
(65992542,963757839)
(963566107,849477578)
(317052188,206776213)
(920703175,515636559)
第2个投票方输入自己的投票值: 1
第2个投票方的投票值的多项式及秘密份额:
f2=1+748516802x^1+845626851x^2
(65992542,517805153)
(963566107,645171488)
(317052188,396210146)
(920703175,883486649)
第3个投票方输入自己的投票值: 0
第3个投票方的投票值的多项式及秘密份额:
f3=0+669433281x^1+234596864x^2
(65992542,57560732)
(963566107,49069945)
(317052188,352907520)
(920703175,449077384)
参与重构秘密的参与者: [3, 2, 0]
得票结果为: 1
```